

Rulmanlarda Erken Arıza Uyarısı

NASA/IMS arızaya kadar çalıştırma (run-to-failure) rulman veri setinde gözetimsiz anomali tespiti ve erken uyarı

Teknik Rapor

Ecem Aydemir

Ekim 2026

Özet

Yuvarlanmalı rulmanlar, döner makinelerdeki arızaların en yaygın kaynaklarından biridir ve uygulamada arıza etiketleri (“bu rulman şu tarihte bozulmaya başladı” diyen kayıtlar) neredeyse hiçbir zaman bulunmaz. Bu rapor, bir makinenin ömrünün ilk kısmından *sağlıklı* titreşimin nasıl görüldüğünü öğrenen ve bir rulman bu durumdan uzaklaştığında kararlı (debounce edilmiş, yani tek bir gürültülü ölçüme kapılmayan) bir alarm üreten, uçtan uca ve yeniden üretilebilir bir erken uyarı sistemini anlatmaktadır. Sistem, saniyede 20 000’den fazla ölçüm alınarak (20 kHz) kaydedilmiş her bir saniyelik titreşim kaydını fiziksel anlamı olan 18 durum göstergesine (zaman alanı istatistikleri, frekans bandı enerjileri ve rulman arıza frekanslarındaki zarf spektrumu enerjisi) indirger; dört dedektörü (RMS eşiği temel modeli, Mahalanobis uzaklığı, Isolation Forest ve LSTM otokodlayıcı) karşılaştırır ve zarf spektrumuna dayalı bir arıza tipi teşhisi ekler. Geliştirme testinde (IMS test 2) tüm dedektörler, arızalanan rulmanda testin bitiminden **yaklaşık 74 saat önce (deney süresinin %45’i)** kalıcı bir alarm verir ve arızayı doğru biçimde **dış bilezik** hasarı olarak teşhis eder. Ayrılmış (held-out, yani ayar yaparken hiç bakılmamış) iki testte ise sonuçlar, uyarı süresi ile yanlış alarm sayısı arasındaki açık bir ödünleşimi ve dürüst bir sınırlamayı ortaya koyar: makine duruşlarından sonra değişen çalışma koşulları anomali gibi görünmektedir. Çalışma; tek komutla çalışan bir veri işleme hattı, birim testleri, açıklamalı bir notebook ve etkileşimli bir Streamlit paneli içeren açık kaynaklı bir Python paketi olarak sunulmaktadır.

İçindekiler

Bu Rapor Nasıl Okunmalı?	3
1 Giriş	4
1.1 Problem	4
1.2 Tek paragrafta yaklaşım	4
1.3 Katkılar	4
1.4 Önceki bir denemeye ilişkisi	5
2 Teorik Arka Plan	6
2.1 Yuvarlanmalı rulmanlar ve hasar tipleri	6
2.2 Titreşim durum göstergeleri	8
2.3 Gözetimsiz anomali tespiti	9
3 Veri Seti	10

4	Kullanılan Araçlar ve Ortam	11
5	Yöntem	12
5.1	Öznitelik çıkarımı	13
5.2	Sinyal düzeyinde örnekleme	15
5.3	Ön işleme ve veri bölmeleri	18
5.4	Dedektörler	19
5.5	Puandan alarma	21
5.6	Teshis	22
5.7	Değerlendirme protokolü	23
5.8	Tasarım kararları	24
6	Sonuçlar	24
6.1	Ana sonuçlar	24
6.2	Test 2 (geliştirme kümesi)	26
6.3	Test 1 (ayrılmış)	29
6.4	Test 3 (ayrılmış)	32
6.5	Bulguların özeti	35
7	Etkileşimli Panel (Dashboard)	35
8	Yazılım Mühendisliği ve Yeniden Üretilebilirlik	38
8.1	Depo yapısı	38
8.2	Yapılandırma	38
8.3	Çalıştırma	39
8.4	Testler	39
8.5	Sürüm kontrolü	40
9	Tartışma ve Sınırlamalar	40
10	Sonuç ve Gelecek Çalışmalar	41
A	Sözlük: Temel Kavramlar	41

Bu Rapor Nasıl Okunmalı?

Hikâye. Bir doktor düşünün. Size her gün aynı saatte stetoskopla kalbinizi dinliyor. İlk birkaç hafta hiçbir şikâyetiniz yok; doktor bu sürede *sizin* sağlıklı kalp sesinizin nasıl olduğunu ezberliyor. Bir gün ses biraz değişiyor. Değişiklik henüz sizin fark edeceğiniz kadar büyük değil, ama doktor için “her zamankinden farklı”. Doktor bir kez tuhaf bir ses duydu diye panik yapmıyor; birkaç gün üst üste aynı değişikliği duyunca sizi uyarıyor. Üstelik sesin *türüne* bakarak sorunun kalbin hangi bölümünde olabileceğini de tahmin ediyor.

Bu proje aynı işi bir makine için yapar. Fabrikalardaki motorların, pompaların, fanların içinde dönen mili taşıyan küçük parçalar vardır: **rulmanlar**. Arabanızın tekerleğinden bir uğultu gelmeye başlaması çoğu zaman bir rulmanın bozulmaya başladığının işaretidir; ses, parça tamamen kırılmadan günler hatta haftalar önce ortaya çıkar. Rulmanın üstüne yapıştırılan küçük bir sensör (ivmeölçer) makinenin titreşimini, yani “sesini”, sürekli kaydeder. Bizim sistemimiz:

1. makinenin ilk, sağlıklı döneminde titreşimin nasıl olduğunu öğrenir (doktorun ilk haftaları);
2. sonra her yeni ölçümü bu “normal” ile karşılaştırıp bir *tuhaflik puanı* verir;
3. puan birkaç ölçüm üst üste belirlenen sınırı aşınca alarm verir (tek bir tuhaf ölçüme kapılmaz);
4. alarm verdiğinde, titreşimin ritmine bakarak hasarın rulmanın hangi parçasında olduğunu tahmin eder.

Neden önemli? Bir rulman aniden kırılırsa makine plansız durur, üretim kaybedilir ve bazen başka parçalar da zarar görür. Çok erken değiştirilirse hâlâ kullanılabilecek bir parça çöpe gider. İdeal olan, bozulmanın başladığını *erken* fark edip onarımı uygun bir zamana planlamaktır. Zorluk şu: gerçek fabrikalarda “bu rulman şu gün bozulmaya başladı” diye etiketlenmiş geçmiş veri neredeyse hiç yoktur. Bu yüzden sistemimiz arızanın neye benzediğini *hiç görmeden*, yalnızca normalin neye benzediğini öğrenerek çalışır.

Veri. NASA’nın herkese açık bir veri setini kullandık: bir test makinesinde dört rulman bozulana kadar haftalarca döndürülmüş ve her 10 dakikada bir saniyelik titreşim kaydedilmiştir. Üç ayrı deney (test 1, test 2, test 3) vardır. Sistemin ayarlarını yalnızca test 2 üzerinde yaptık; test 1 ve test 3’ü “sınav soruları” gibi sona sakladık.

Ana sonuç tek cümlede. En temiz deneyde sistem, rulman bozulmadan yaklaşık üç gün (74 saat) önce kalıcı bir alarm verdi ve hasarın rulmanın dış halkasında (dış bilezikte) olduğunu doğru söyledi; daha zor iki deneyde ise “erken uyarmak” ile “boş yere alarm vermemek” arasında bir seçim yapmak gerektiğini ve makine durdurulup yeniden çalıştırıldığında sistemin şaşırabildiğini dürüstçe gösterdi.

Okuma haritası.

- **Hiç teknik bilginiz yoksa:** bu bölümü, Giriş’i (Bölüm 1), her bölümdeki mavi çerçeveli “**Basitçe**” kutularını, her grafiğin altındaki gri çerçeveli “**Bu grafik nasıl okunur?**” kutularını ve grafik yorumlarındaki **Ne görüyoruz?** / **Ne anlama geliyor?** / **Neden önemli?** kısımlarını okuyun. Sonuçlar (Bölüm 6), Tartışma (Bölüm 9) ve Sonuç (Bölüm 10) bölümlerindeki metin de sade tutulmuştur.
- **Göz gezdirebileceğiniz kısımlar:** formüller, kod blokları (gri arka planlı, satır numaralı kutular), Veri Seti bölümündeki dosya ayrıntıları ve Yazılım Mühendisliği bölümü. Her formülün ve her kod bloğunun hemen altında ne yaptığı günlük dille anlatılmıştır; formülün kendisini anlamadan o açıklamayı okumanız yeterlidir.

- **Bilmediğiniz bir kelimeye takılırsanız:** rapordaki tüm teknik terimler sonda, Ek A'deki “Sözlük: Temel Kavramlar” bölümünde sade dille ve örneklerle açıklanmıştır.

1 Giriş

1.1 Problem

Kestirimci bakımın (bir parçayı takvime göre değil, durumuna bakarak tam gerektiğinde onarma yaklaşımının) amacı bir makineyi arızalanmadan *kısa bir süre önce* onarmaktır: bileşenin ömrünün büyük kısmını kullanacak kadar geç, müdahaleyi planlayacak kadar erken. Döner makineler için en zengin bilgi kaynağı titreşimdir ve en sık arızalanan bileşen rulmanlardır. Sensörlü bir makine sürekli titreşim verisi üretir; ancak bu veri neredeyse hiçbir zaman “bu rulman şu an bozulmaya başladı” diyen etiketlerle gelmez. Arızalar nadir, pahalı ve birbirinden farklıdır; bu nedenle bilinen arızaları sınıflandırmak üzere eğitilmiş bir model yeni bir makine için genellikle mevcut değildir.

Bu projenin cevapladığı soru şudur:

Yalnızca makinenin sağlıklı olduğu varsayılan bir dönemin verisini kullanarak, bir operatörü bir rulmanın arızalanmaya başladığı konusunda erken, güvenilir ve az sayıda yanlış alarmla uyarabilir ve arızanın muhtemel tipini söyleyebilir miyiz?

Basitçe

Elimizde makinenin titreşim kayıtları var ama “arıza burada başladı” diye işaretlenmiş hiçbir örnek yok. Soru şu: sadece makinenin sağlıklı hâlini öğrenerek, bozulma başladığında bunu fark edip operatöre zamanında, gereksiz yere bağırıp çağırmadan haber verebilir miyiz? Ve “sorun büyük ihtimalle şu parçada” diyebilir miyiz?

1.2 Tek paragrafta yaklaşım

Her titreşim kaydı kompakt bir durum göstergesi vektörüne (makinenin o anki durumunu özetleyen 18 sayılı bir listeye) dönüştürülür. Her deneyin ilk %25’lik kısmı sağlıklı kabul edilir ve bir anomali dedektörünü eğitmek için kullanılır; sonraki %10’luk kısım alarm eşiğini belirler. Bu noktadan sonra dedektör her yeni kaydı puanlar; son 6 puanın en az 4’ü eşiği aştığında alarm verilir ve zarf enerjisi en çok artan arıza frekansı muhtemel arıza tipini belirtir. Sistem üç adet arızaya kadar çalıştırma deneyinde, alarm ile testin bitişi arasındaki saat sayısı (*uyarı süresi*, lead time) ve makine henüz sağlıklıken verilen alarm sayısı (*yanlış alarm*) ile değerlendirilmiştir.

Basitçe

Her bir saniyelik titreşim kaydını 18 sayıyla özetliyoruz (ne kadar güçlü titriyor, ne kadar “vuruntulu”, hangi seslerde yoğunlaşıyor gibi). Deneyin ilk dörtte birinde makinenin sağlıklı olduğunu varsayıp sisteme “normal budur” diye öğretiyoruz. Sonraki kısa dönemde sistemin normalde ne kadar dalgalandığına bakıp bir sınır çiziyoruz. Sonra her yeni kayda bir “tuhaflik puanı” veriyoruz; son 6 kaydın 4’ü sınırı aşarsa alarm çalıyor. Sistemi iki soruyla notluyoruz: rulman kırılmadan kaç saat önce uyardı (ne kadar çok o kadar iyi) ve makine sağlamken kaç kez boşuna alarm verdi (ne kadar az o kadar iyi).

1.3 Katkıları

- IMS veri seti için sızıntısız (leakage-free; sistemin eğitim sırasında “geleceği” yanlışlıkla görmediği), zaman sıralı bir değerlendirme protokolü; tüm üç teste ve arızalanan tüm

rulmanlara uygulanmıştır. Test 2 geliştirme için kullanılmış, test 1 ve test 3 ayrılmış (held-out) olarak tutulmuştur.

- Test düzeneğindeki rulmanın gerçek geometrisi için hesaplanmış, arıza frekanslarında zarf analizi içeren fiziksel temelli öznitelikler (ölçülen sinyalden hesaplanan, fiziksel anlamı olan özet sayılar).
- Klasik bir temel model, iki klasik makine öğrenmesi dedektörü ve bir derin öğrenme dizi modelinin aynı veri bölmeleri, eşikler ve alarm mantığı altında karşılaştırılması.
- Operatöre yönelik alarm mantığı (debounce, kalıcılık, ilk alarm veren rulmanla kaynak tespiti ve arıza teşhisi) ve bir testi canlı veri akışı gibi yeniden oynatan etkileşimli bir panel.
- Yeniden üretilebilir bir yazılım paketi: yapılandırma dosyası, Makefile, birim testleri ve depoya eklenmiş öznitelik tabloları (3 MB). Böylece tüm sonuçlar 6 GB ham veriyi indirmeden yeniden üretilebilir.

Basitçe

Bu çalışmanın ortaya koyduğu beş şey: (1) sistemi “kopya çekmeden”, yani geleceği görmeden sınavan dürüst bir test düzeni; (2) rulmanın fiziğine dayanan, anlamı açık ölçümler; (3) basitten karmaşığa dört farklı yöntemin aynı kurallarla yarışdırılması; (4) bir operatörün gerçekten kullanabileceği alarm mantığı ve ekran; (5) herkesin kendi bilgisayarında aynı sonuçları tekrar elde edebileceği düzenli bir yazılım.

1.4 Önceki bir denemeye ilişkisi

Bu proje, aynı veri setinde bir LSTM otokodlayıcı eğiten daha önceki bir notebook’un (2025) yeniden ele alınmasıdır. O çalışmanın incelenmesi, sonuçlarını geçersiz kılan birkaç yönetsel sorunu ortaya çıkardı. Bu sorunlar bu veri setinde sık yapılan hatalar olduğu ve Bölüm 5.8’deki tasarım kararlarını doğrudan yönlendirdiği için burada listelenmiştir:

1. **Model arızalı veriyle eğitilmişti.** Eğitim kümesi deneyin *son* bir milyon örneği idi; yani rulmanın bozulduğu dönem. Aynı veri daha sonra test için de kullanılmıştı.
2. **Yanlış rulman izleniyordu.** Test 1’in yalnızca ilk kanalı (rulman 1) kullanılmıştı; oysa test 1’de arızalanan rulmanlar 3 ve 4’tür, rulman 1 sağlam kalır.
3. **Zaman bilgisi kaybolmuştu.** 2 156 dosyanın tamamı uç uca eklenip 50 örneklik (2,5 ms) pencerelere bölünmüştü. Zaman eksenini olmadan “arızadan kaç saat önce uyardı?” sorusu cevaplanamaz.
4. **Eğitim ve doğrulama arasında sızıntı.** Üst üste binen pencereler eğitim/doğrulama bölmesinden önce karıştırılmıştı; bu nedenle doğrulama kaybı olduğundan iyi görünüyordu.

Önceki çalışmanın temel fikri (“normal”i bir otokodlayıcıyla öğrenip yeniden oluşturma hatasını anomali puanı olarak kullanmak) korunmuştur; bu rapordaki LSTM otokodlayıcı dedektörü, aynı fikrin doğru kurulmuş hâlidir.

Basitçe

Bu proje, daha önce yapılmış ama hatalı kurulmuş bir denemenin düzeltilmiş hâlidir. Eski denemede sistem (1) “sağlıklı” diye bozuk makinenin verisiyle eğitilmişti; bu, bir doktora sağlıklı kalp sesi diye hasta birinin kalbini dinletmek gibidir; (2) bozulmayan bir rulman izleniyordu; (3) kayıtların ne zaman alındığı bilgisi atılmıştı, bu yüzden “kaç saat önce uyardı?” sorusu sorulamıyordu; (4) sınav soruları çalışma kâğıtlarına karışmıştı, yani sistem sınavda gördüğü soruları önceden görmüştü ve olduğundan başarılı görünüyordu. Bu rapordaki tüm

tasarım bu dört hatadan kaçınmak üzerine kurulmuştur.

2 Teorik Arka Plan

Basitçe

Bu bölüm, sistemin dayandığı üç temel fikri anlatır: (1) bir rulman nasıl bozulur ve bozulunca titreşimde nasıl bir “iz” bırakır; (2) bir saniyelik titreşimi hangi sayılarla özetleyebiliriz; (3) “normalden ne kadar farklı?” sorusunu soran dört yöntem nasıl çalışır. Formülleri atlayabilirsiniz; her birinin altında ne anlattığı günlük dille açıklanmıştır.

2.1 Yuvarlanmalı rulmanlar ve hasar tipleri

Bir yuvarlanmalı rulman; mile takılı iç bilezik, gövdeye takılı dış bilezik, yuvarlanma elemanları (bilye veya makaralar) ve bu elemanları birbirinden ayıran kafesten oluşur. Yorulma, kirlenme veya yetersiz yağlama bu yüzeylerden birinde lokal bir hasar (oyuk ya da çatlak) oluşturur. Bir yuvarlanma elemanı hasarın üzerinden her geçtiğinde kısa bir mekanik darbe oluşur. Bu darbeler yalnızca rulman geometrisine ve mil hızına bağlı bir frekansla tekrarlanır ve rulman ile gövdenin, genellikle kHz mertebesindeki (saniyede binlerce kez salınan) yapısal rezonanslarını uyarır.

Basitçe

Rulmanı iç içe geçmiş iki halka gibi düşünün: içteki halka (iç bilezik) dönen mile, dıştaki halka (dış bilezik) sabit gövdeye takılıdır. Aralarında küçük silindirler (makaralar) yuvarlanır; bir kafes bu makaraların birbirine çarpmadan eşit aralıklı durmasını sağlar. Halkalardan birinin yüzeyinde küçük bir çukur oluşursa, her makara o çukurun üzerinden geçerken bir “tık” sesi çıkar; tıpkı yoldaki bir çukurdan geçen arabanın tekerleği gibi. Bu tıklar, rulmanın ölçülerine ve milin dönüş hızına bağlı, çok düzenli bir ritimle tekrarlanır. Her tık aynı zamanda rulman gövdesini bir çan gibi “çınlatır” (rezonans). Sistemimizin temel fikri bu ritmi yakalamaktır: ritmin hızı, çukurun hangi parçada olduğunu söyler.

Mil frekansı f_r , n adet çapı d olan yuvarlanma elemanı, hatve çapı D ve temas açısı ϕ için karakteristik arıza frekansları şunlardır:

$$\text{BPFO} = \frac{n}{2} f_r \left(1 - \frac{d}{D} \cos \phi \right) \quad (\text{dış bilezik geçiş frekansı}) \quad (1)$$

$$\text{BPFI} = \frac{n}{2} f_r \left(1 + \frac{d}{D} \cos \phi \right) \quad (\text{iç bilezik geçiş frekansı}) \quad (2)$$

$$\text{BSF} = \frac{D}{2d} f_r \left(1 - \left(\frac{d}{D} \cos \phi \right)^2 \right) \quad (\text{yuvarlanma elemanı dönme frekansı}) \quad (3)$$

$$\text{FTF} = \frac{f_r}{2} \left(1 - \frac{d}{D} \cos \phi \right) \quad (\text{kafes frekansı}) \quad (4)$$

Formüldeki simgeler.

- f_r : milin saniyede kaç tur attığı (dönme frekansı, Hz cinsinden).
- n : bir sıradaki yuvarlanma elemanı (makara) sayısı.
- d : bir makaranın çapı; D : hatve çapı, yani makaraların merkezlerinden geçen dairenin çapı.
- ϕ : temas açısı, makaranın halkaya hangi eğimle bastığı; $\cos \phi$ bu eğimin etkisini hesaba katan bir düzeltme çarpanıdır.

- $\frac{d}{D} \cos \phi$: makaranın rulmana göre ne kadar “büyük” olduğunu anlatan küçük bir oran (burada yaklaşık 0,11).

Sezgisel anlamı. Makaralar mil ile birlikte, ama milden biraz daha yavaş, halkalar arasında dolaşır. Dış bilezikteki sabit bir çukurun üzerinden saniyede kaç makara geçtiğini BPFO verir: kabaca “makara sayısının yarısı $\times$ mil hızı”, makaraların yavaş dolaşması nedeniyle biraz azaltılmış hâli (parantezdeki eksi). İç bilezik mil ile birlikte döndüğü için çukura makaralar daha sık rastlar; BPFI’de parantez içi bu yüzden artıdır. BSF, bir makaranın kendi eksenini etrafında dönme hızıdır (makaradaki bir çukur halkalara bu ritimle çarpar). FTF ise kafesin, yani makara grubunun mil etrafında dolaşma hızıdır; mil hızının yarısından biraz azdır. Önemli nokta: bu dört sayı yalnızca rulmanın ölçülerinden ve hızından hesaplanır; titreşimde hangi ritmin belirdiğine bakarak hasarlı parçayı söyleyebiliriz.

IMS test düzeneği, sıra başına $n = 16$ makaralı, $d = 0,331$ inç, $D = 2,815$ inç ve $\phi = 15,17^\circ$ olan Rexnord ZA-2115 çift sıralı rulmanlar kullanır ve 2000 dev/dk ($f_r = 33,3$ Hz) hızla döner. Elde edilen frekanslar Tablo 1’de, kod karşılığı Kod 1’de verilmiştir.

Tablo 1: ZA-2115 rulmanının 2000 dev/dk’daki karakteristik arıza frekansları.

Frekans	Değer [Hz]	İşaret ettiği hasar
BPFO	236,4	dış bilezik
BPFI	296,9	iç bilezik
BSF	139,9	yuvarlanma elemanı
FTF	14,8	kafes

Tabloyu şöyle okuyabilirsiniz: dış bilezikte bir çukur varsa titreşimde saniyede yaklaşık 236 kez tekrarlanan bir “tık” ritmi görmeyi bekleriz; iç bilezikte bir çukur varsa bu ritim saniyede yaklaşık 297’dir. Aşağıdaki kod, bu hesabı bilgisayarda yapan kısımdır: rulmanın ölçülerini alıp yukarıdaki dört formülü uygular.

Kod 1: Rulman geometrisinden arıza frekansları (src/bearing_efw/dataset.py).

```

1 FS = 20_480 # sampling rate [Hz]
2 SHAFT_HZ = 2000 / 60.0
3 N_ROLLERS, ROLLER_D, PITCH_D, CONTACT_ANGLE_DEG = 16, 0.331, 2.815, 15.17
4
5 def fault_frequencies(fr: float = SHAFT_HZ) -> dict[str, float]:
6     """Characteristic defect frequencies [Hz] for the ZA-2115 bearing."""
7     ratio = ROLLER_D / PITCH_D * np.cos(np.deg2rad(CONTACT_ANGLE_DEG))
8     return {
9         "bpfo": N_ROLLERS / 2 * fr * (1 - ratio), # outer race
10        "bpfi": N_ROLLERS / 2 * fr * (1 + ratio), # inner race
11        "bsf": PITCH_D / (2 * ROLLER_D) * fr * (1 - ratio**2), # rolling element
12        "ftf": fr / 2 * (1 - ratio), # cage
13    }

```

Kod ne yapıyor? Satır 1, sensörün saniyede kaç ölçüm aldığını (20480) kaydeder. Satır 2, dakikada 2000 devri saniyede tur sayısına çevirir ($2000/60 \approx 33,3$). Satır 3 rulmanın ölçülerini (makara sayısı, makara çapı, hatve çapı, temas açısı) yazar. Satır 7, formüllerde tekrar tekrar geçen $\frac{d}{D} \cos \phi$ oranını bir kez hesaplar (açı önce dereceden radyana çevrilir). Satır 9–12 dört formülü birebir uygular ve sonuçları adlarıyla birlikte döndürür; satır sonlarındaki yorumlar hangi frekansın hangi parçaya ait olduğunu söyler.

2.2 Titreşim durum göstergeleri

Basitçe

Bir saniyelik kayıta 20 480 sayı vardır; bunların hepsine tek tek bakmak hem zor hem anlamsızdır. Bu yüzden her kaydı, bir doktorun tahlil sonucundaki değerler gibi, birkaç anlamlı özet sayıya indiriyoruz. Üç tür özet kullanıyoruz: (1) titreşimin genel gücü ve ne kadar “vuruntulu” olduğu; (2) titreşimin hangi seslerde, yani bas tonlarda mı tiz tonlarda mı yoğunlaştığı; (3) hasar ritimlerinden (BPFO, BPFI, BSF) birinin titreşimde belirip belirmediği. Bu özet sayılara “öznitelik” ya da “durum göstergesi” diyoruz.

Zaman alanı istatistikleri. Ortalaması sıfıra çekilmiş bir $x_1, \dots, x_N$ kaydı için:

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_i x_i^2}, \quad \text{basıklık (kurtosis)} = \frac{\frac{1}{N} \sum_i x_i^4}{\text{RMS}^4}, \quad \text{tepe faktörü} = \frac{\max_i |x_i|}{\text{RMS}}. \quad (5)$$

Formüldeki simgeler. $x_1, x_2, \dots, x_N$ kayıttaki ardışık titreşim ölçümleridir (burada $N = 20\,480$). x_i bunlardan i 'inci olanıdır. $\sum_i$ “hepsini topla” demektir; $\frac{1}{N} \sum_i$ ise “hepsinin ortalamasını al”. x_i^2 ve x_i^4 ölçümün karesi ve dördüncü kuvvetidir. $|x_i|$ işaretten bağımsız büyüklüktür (eksi değerleri artıya çevirir) ve $\max_i |x_i|$ kayıttaki en büyük sapma, yani tepe değeridir.

Sezgisel anlamı.

- **RMS** sinyalin ortalama “gücü”dür: her ölçümün karesini alıp ortalayıp karekökünü alırsınız. Titreşim büyüdükçe artar; bir hoparlörün ses seviyesi gibi düşünebilirsiniz.
- **Basıklık** sinyalde seyrek ama çok büyük sıçramalar olup olmadığını ölçer. Dördüncü kuvvet büyük değerleri aşırı büyüttüğü için, arada bir gelen sert “tık”lar basıklığı hızla yükseltir. Düzgün, hışırtı gibi bir gürültüde değeri 3’tür; 3’ün çok üstü “sinyalde darbeler var” demektir.
- **Tepe faktörü** en büyük sıçramanın ortalama güce oranıdır: “en sert vuruş, ortalamanın kaç katı?”

RMS toplam titreşim enerjisini ölçer ve endüstriyel standartların çoğunun (ör. ISO 10816/20816) kullandığı göstergedir. Basıklık Gauss gürültüsü (rastgele, düzgün hışırtı benzeri gürültü) için 3’tür ve sinyal keskin darbeler içerdiğinde büyür; bu nedenle enerji artmadan önce bile erken, lokal hasarlara duyarlıdır. Tepe (crest), biçim ($\text{RMS}/\overline{|x|}$) ve darbe ($\max |x|/\overline{|x|}$) faktörleri benzer darbesellik ölçüleridir ($\overline{|x|}$, ölçümlerin büyüklüklerinin ortalamasıdır); çarpıklık (skewness) asimetriyi, yani sinyalin bir yöne diğerinden daha fazla sapıp saptığını yakalar.

Spektral bant enerjileri. Güç spektrumunun (titreşim enerjisinin frekanslara, yani bas ve tiz tonlara dağılımının) altı banttaki (0–1, 1–2, 2–4, 4–6, 6–8 ve 8–10,24 kHz) payı ile spektral ağırlık merkezi (enerjinin “ortalama frekansı”), enerjinin *nerede* toplandığını tanımlar. Gelişen bir hasar enerjisi yüksek frekanslı rezonanslara doğru kaydırır.

Zarf (envelope) analizi. Hasar darbeleri kısa sürelidir ve yüksek frekanslı rezonansları uyarır; bunların *tekrarlanma hızı* (ör. BPFO) rezonansın altında gizli kaldığı için ham spektrumda görünmez. Zarf analizi bu bilgiyi üç adımda geri kazanır [3]:

1. sinyal rezonans bölgesinde bant geçiren filtreden geçirilir (burada 2–8 kHz, 4. dereceden Butterworth, sıfır faz; yani yalnızca bu aralıktaki sesleri geçiren, sinyali zamanda kaydırmayan bir süzgeç),
2. analitik sinyalin (Hilbert dönüşümü) genliği alınır; bu, genlik modülasyonlu rezonansın zarfıdır,

- zarfın spektrumu hesaplanır; lokal bir hasar, karakteristik frekansında ve harmoniklerinde (bu frekansın 2, 3, ... katlarında) tepe üretir.

Burada kullanılan öznitelik, zarf spektrumu enerjisinin (5–1000 Hz) BPFO, BPFI ve BSF’nin ilk üç harmoniğinin ± 3 Hz çevresine düşen payıdır.

Basitçe

Zarf analizini bir benzetmeyle anlatalım. Uzaktan, saniyede birkaç kez çalan bir zil düşünün. Zilin sesi tizdir (yüksek frekanslı “çınlama”), ama bizi ilgilendiren zilin *ne sıklıkla* çaldığıdır. Bunu bulmak için önce sadece zilin tonunu duyacak şekilde diğer sesleri süzeriz (bant geçiren filtre). Sonra çınlamanın kendisini değil, sesin “açılıp kapanmasını”, yani her çalışın oluşturduğu kabarıklığın dış hatlarını çizeriz (zarf; Hilbert dönüşümü bu dış hattı çizmenin matematiksel yoludur). Son olarak bu dış hattın hangi ritimle tekrarlandığına bakarız (zarf spektrumu). Rulmanda “zil” gövdenin rezonansıdır, “çalma sıklığı” ise hasar ritmidir (ör. dış bilezik için saniyede 236 kez). Bizim öznitelik, zarfın enerjisinin ne kadarının tam bu hasar ritimlerinde (ve katlarında) toplandığını ölçer.

2.3 Gözetimsiz anomali tespiti

Arıza etiketi kullanılmadığı için tüm dedektörler *tek sınıflı* (one-class) veya *yenilik* (novelty) dedektörleridir: yalnızca sağlıklı veriyle eğitilirler ve yeni bir gözlem sağlıklı veriye ne kadar az benziyorsa o kadar büyük bir puan üretirler.

Basitçe

“Gözetimsiz” öğrenme, sisteme doğru cevapların gösterilmediği öğrenmedir: kimse “bu arıza, bu değil” demez. Sisteme sadece sağlıklı makinenin ölçümleri gösterilir ve o bunlardan “normal”in neye benzediğini çıkarır. Sonra her yeni ölçüme “bu, normale ne kadar benzemiyor?” sorusunun cevabı olan bir puan verir. Aşağıda bu puanı hesaplayan dört farklı yöntem var; her biri “normalden farklılığı” başka bir açıdan ölçer.

Mahalanobis uzaklığı. Sağlıklı öznitelik vektörlerinin ortalaması μ ve kovaryansı Σ ile

$$d_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu)}. \quad (6)$$

Formüldeki simgeler. $\mathbf{x}$ yeni bir kaydın öznitelik listesidir (ör. 18 sayı). μ sağlıklı dönemdeki her öznitelik ortalamasıdır, yani “tipik sağlıklı kayıt”. $\mathbf{x} - \mu$ yeni kaydın bu tipik kayıttan her öznitelikte ne kadar saptığıdır. Σ (kovaryans) sağlıklı dönemde özniteliklerin ne kadar dalgalandığını ve hangilerinin birlikte arttığını (birbiriyle ilişkili olduğunu) özetleyen bir tablodur. Σ^{-1} bu tablonun tersidir ve sapmaları “normal dalgalanmaya göre” ölçeklemek için kullanılır. $^\top$ ve çarpımlar tüm öznitelikleri tek bir sayıda toplamak için kullanılan matris işlemleridir.

Sezgisel anlamı. Bu, “yeni ölçüm, sağlıklı ölçümler bulutunun merkezinden kaç adım uzakta?” sorusunun cevabıdır; ama adımlar akıllıca ölçülür. Normalde çok dalgalanan bir öznitelikteki sapma az, normalde hiç kıpırdamayan bir öznitelikteki sapma çok önemlidir. Ayrıca normalde birlikte hareket eden iki öznitelik (ör. RMS ve tepe değeri) birbirinden ayrılırsa bu da tuhaf sayılır. Bir benzetme: boyu uzun birinin kilosunun da fazla olması olağandır; ama çok uzun ve çok hafif biri dikkat çeker.

Bu ölçü öznitelikler arasındaki korelasyonları (ör. RMS ve tepe değeri) hesaba katar. Pek çok öznitelik birbiriyle ilişkili ve eğitim kümesi küçük olduğundan Σ , örnek kovaryansı ölçeklenmiş birim matrise doğru çekerek iyi koşullu tutan Ledoit–Wolf büzülme (shrinkage) yöntemiyle kestirilir [5]. (Sade dille: az veriyle hesaplanan ilişki tablosu güvenilirmez ve uç değerler üretebilir; Ledoit–Wolf

bu tabloyu “öznitelikler birbirinden bağımsızdır” diyen basit bir tabloya doğru biraz yumuşatarak daha kararlı hâle getirir.)

Isolation Forest. Gözlemleri rastgele bölmelerle yalıtın rastgele ağaçlardan oluşan bir topluluktur [4]. Anomaliler az sayıda ve farklı oldukları için daha az bölmeye yalıtılır; puan ortalama yol uzunluğundan türetilir. (Benzetme: bir kalabalıkta “boyu 1,70’ten uzun mu?”, “şapka takıyor mu?” gibi rastgele sorularla bir kişiyi diğerlerinden ayırmaya çalıştığınızı düşünün. Herkese benzeyen birini ayırmak için çok soru gerekir; turuncu kostümlü birini ise bir iki soruyla ayırırsınız. Az soruyla ayrılan gözlem tuhaftır. “Orman” bu oyunun yüzlerce kez farklı rastgele sorularla oynanmasıdır.)

LSTM otokodlayıcı. Otokodlayıcı girdisini düşük boyutlu bir gizli koda sıkıştırır ve yeniden oluşturur. Yalnızca sağlıklı veriyle eğitildiğinde sağlıklı girdileri iyi, tanımadığı girdileri kötü yeniden oluşturur; bu nedenle W kayıt ve p öznitelikten oluşan bir pencere üzerindeki yeniden oluşturma hatası

$$e_t = \frac{1}{Wp} \sum_{k=0}^{W-1} \sum_{j=1}^p (x_{t-k,j} - \hat{x}_{t-k,j})^2 \quad (7)$$

bir anomali puanıdır [7]. Uzun kısa süreli bellek (LSTM) katmanları [6] modelin pencere içindeki zamansal değişimi kullanmasını sağlar.

Formüldeki simgeler. t şu anki kaydın sıra numarasıdır. W pencere uzunluğu, yani geriye doğru kaç kayda birlikte bakıldığıdır (burada 12 kayıt, yaklaşık iki saat). p öznitelik sayısıdır. k “kaç kayıt geriye” ($0 = \text{şimdiki kayıt}$), j “kaçıncı öznitelik” anlamına gelir. $x_{t-k,j}$ gerçek değer, $\hat{x}_{t-k,j}$ ise modelin yeniden oluşturduğu (tahmin ettiği) değerdir. Farkın karesi alınıp tüm kayıtlar ve öznitelikler üzerinden ortalananır ($\frac{1}{Wp} \sum \sum$).

Sezgisel anlamı. Otokodlayıcı, bir resmi küçük bir özete sıkıştırıp sonra o özette resmi yeniden çizmeye çalışan bir ressam gibidir. Ressam yalnızca sağlıklı makine “resimleri” üzerinde çalışmıştır; tanıdık bir resmi neredeyse kusursuz yeniden çizer, hiç görmediği türden bir resmi ise beceremez. e_t , son iki saatlik pencerede gerçek değerlerle yeniden çizilenler arasındaki ortalama farktır: ne kadar büyükse, makinenin durumu öğrenilen “normal”e o kadar az benzer.

RMS eşiği. Her bakım mühendisinin ilk deneyeceği temel yöntem: toplam titreşim seviyesi sağlıklı aralığının üstüne çıktığında alarm. Daha karmaşık bir yöntemin karmaşıklığına değmesi için bunu geçmesi gerekir.

3 Veri Seti

Basitçe

Bu bölüm, kullandığımız verinin nereden geldiğini ve neye benzediğini anlatır. Kısaca: bir laboratuvar da tek bir mil üzerinde dört rulman, ağır yük altında, biri bozulana kadar günlerce döndürülmüş. Her rulmanın üstündeki bir sensör, her 10 dakikada bir, bir saniye boyunca titreşimi kaydetmiş. Üç ayrı deney yapılmış; her birinin sonunda farklı bir rulman, farklı bir yerinden bozulmuş. Bu bölümün sonunda veriyi hazırlarken karşılaştığımız pratik sorunlar (bozuk kayıtlar, kayıt boşlukları) da listelenmiştir.

IMS rulman veri seti [1, 2], Cincinnati Üniversitesi’ndeki NSF I/UCR Akıllı Bakım Sistemleri Merkezi (Center for Intelligent Maintenance Systems) tarafından üretilmiş ve NASA Prognostik Mükemmeliyet Merkezi (PCoE) tarafından dağıtılmaktadır. Bir AC motorla sabit 2000 dev/dk’da döndürülen tek bir mil üzerinde dört rulman bulunur ve bir yay mekanizmasıyla 6000 lb radyal yük uygulanır. Rulman gövdelerindeki ivmeölçerler (titreşimi ölçen sensörler; PCB 353B33) bir NI

DAQCard-6062E veri toplama kartıyla 20 kHz’de örneklenir (yani saniyede yaklaşık 20 000 ölçüm alınır). Her 10 dakikada bir (test 1’in başında 5 dakikada bir) kanal başına 20 480 örnekten oluşan bir saniyelik bir kayıt, adı zaman damgası olan bir metin dosyasına yazılır. Her test bir rulman arızalanana kadar sürdürülmüştür.

Tablo 2: Üç arızaya kadar çalıştırma testi. Süre, ölü kayıtlar çıkarıldıktan sonra ölçülmüştür.

Test	Kayıt	Kanal	Süre [sa]	Test sonundaki arıza
test1	2 156	8 (rulman başına x, y)	827,6	rulman 3 iç bilezik, rulman 4 yuvarlanma elemanı
test2	984	4	163,5	rulman 1 dış bilezik
test3	6 324	4	1 073,1	rulman 3 dış bilezik

Tabloyu okumak için: “Kayıt” testte kaç adet bir saniyelik titreşim dosyası olduğunu, “Kanal” kaç sensör kullanıldığını (test 1’de her rulmanda iki sensör, iki farklı yönde ölçüm yapar), “Süre” testin saat cinsinden uzunluğunu (test 3 yaklaşık 44 gün sürmüştür), son sütun ise test bittiğinde hangi rulmanın hangi parçasının hasarlı bulunduğunu gösterir. Bu son sütun, elimizdeki tek “doğru cevap” bilgisidir.

Veriyle çalışırken keşfedilen pratik ayrıntılar.

- Arşiv, içinde bir 7-Zip dosyası, onun içinde de üç RAR dosyası bulunan bir ZIP’tir (yaklaşık 1 GB indirme, açıldığında 6,2 GB, 9 464 kayıt dosyası).
- `3rd_test.rar` dosyası `4th_test/txt` adlı bir klasöre açılır.
- Dosyalar sekmeyle ayrılmış, Windows satır sonlu ve başlıksızdır.
- Test 2 ve test 3’ün son kaydı, kapanıştan sonra kaydedilmiş sifıra yakın bir sinyaldir (RMS $\approx 0,001$ g). Bu “ölü” kayıtlar çıkarılır (herhangi bir kanalda RMS $< 0,01$ g). (Makine durduktan sonra sensör neredeyse hiçbir şey ölçmez; bu kayıtları “rulman birden sessizleşti” diye yorumlamak için atarız.)
- Test 1, 148 saate varan boşluklarla parçalar hâlinde kaydedilmiştir; test 3’te 4,5, 7,5 ve 7,3 saatlik duruşlar vardır (yaklaşık 262., 354. ve 1002. saatlerde). Test 2 kesintisizdir.

İndirme. `scripts/download_data.py`, arşivi NASA deposunun PHM Society aynasından indirir ve tüm katmanları açar (`py7zr` ile `unar` veya `unrar` komutu gerekir). Ham veri Git’e eklenmez; çıkarılmış öznitelikler depoya eklenmiştir.

4 Kullanılan Araçlar ve Ortam

Basitçe

Bu bölüm, projeyi yaparken kullandığımız yazılımların bir listesidir; bir yemek tarifindeki mutfak aletleri listesi gibi. Python programlama dilini, onun üstünde sayılarla çalışmaya, sinyal işlemeye, makine öğrenmesine, grafik çizmeye ve etkileşimli ekran yapmaya yarayan hazır kütüphaneleri kullandık. Önemli bir not: her şey sıradan bir dizüstü bilgisayarda birkaç dakikada çalışır; özel donanım gerekmez.

Tablo 3: Projede kullanılan yazılımlar.

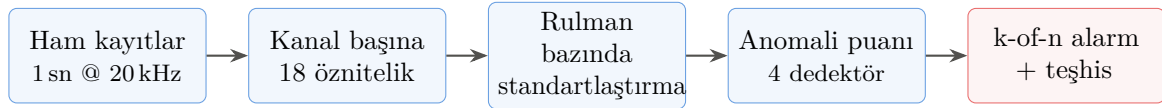
Araç	Görevi
Python 3.10+	uygulama dili
NumPy, pandas	sayısal diziler, öznitelik tabloları, zaman işlemleri
SciPy (signal, stats)	Butterworth filtreleme, Hilbert dönüşümü, basıklık/çarpıklık
scikit-learn	Ledoit–Wolf kovaryansı, Isolation Forest
PyTorch	LSTM otokodlayıcı (CPU yeterlidir)
Matplotlib	README ve bu rapor için statik grafikler
Streamlit, Plotly	etkileşimli yeniden oynatma paneli
PyYAML	deney yapılandırması (configs/default.yaml)
pytest	birim testleri ve panel duman testleri (Streamlit AppTest)
Jupyter / nbconvert	çalıştırılmış açıklamalı notebook
py7zr, unar	iç içe 7z/RAR arşivinin açılması
GNU Make	tek komutluk giriş noktaları (make experiment, make dashboard, ...)
Git, GitHub	sürüm kontrolü, pull request’ler
L ^A T _E X (pdfLaTeX, TikZ, listings)	bu rapor

Tüm hat bir dizüstü bilgisayar işlemcisinde çalışır: 9 464 dosyadan öznitelik çıkarımı 4 süreçle (dört işlemci çekirdeğinin aynı anda çalışmasıyla) yaklaşık 1,5 dakika, dört dedektörün üç testte eğitilip değerlendirilmesi yaklaşık 1,5 dakika sürer.

5 Yöntem

Basitçe

Bu bölüm sistemin adım adım nasıl çalıştığını anlatır. Bir fabrika üretim bandı gibi düşünün: ham titreşim kaydı banda girer; önce 18 özet sayıya çevrilir; sonra her rulmanın kendi “normal”ine göre ölçeklenir; sonra bir dedektör ona bir tuhafılık puanı verir; en sonunda bir alarm kuralı “alarm çılsın mı?” kararını verir ve gerekiyorsa hasarın yerini tahmin eder. Aşağıdaki şekil bu bandı gösterir; alt bölümler her istasyonu sırayla açıklar.



9 464 dosya (6 GB) → 3 MB öznitelik tablosu → her rulman ve kayıt için bir puan ve bir alarm durumu

Şekil 1: Veri işleme hattı.

Bu grafik nasıl okunur?

Şekil soldan sağa okunur; her kutu bir işlem adımı, her ok bir adımın çıktısının bir sonraki adıma verildiğini gösterir. (1) *Ham kayıtlar*: her biri bir saniyelik, saniyede yaklaşık 20 000 ölçüm içeren titreşim dosyaları. (2) *Kanal başına 18 öznitelik*: her dosya her sensör için 18 özet sayıya indirilir. (3) *Rulman bazında standartlaştırma*: sayılar her rulmanın kendi sağlıklı dönemine göre ölçeklenir. (4) *Anomali puanı*: dört dedektörden biri her kayda bir tuhafılık puanı verir. (5) Kırmızı çerçeveli son kutu *alarm kararı ve teşhis*dir. Altındaki gri yazı veri

miktarının nasıl küçüldüğünü özetler: 6 GB'lık ham veri 3 MB'lık bir tabloya, o da her rulman ve her an için tek bir puana ve “alarm var/yok” bilgisine dönüşür.

5.1 Öznitelik çıkarımı

Basitçe

Bu adım, her bir saniyelik kaydı 18 anlamlı sayıya çevirir: 8 tanesi titreşimin gücünü ve vuruntululuğunu, 7 tanesi enerjinin hangi tonlarda yoğunlaştığını, 3 tanesi de dış bilezik, iç bilezik ve makara hasar ritimlerinin ne kadar belirgin olduğunu anlatır. Neden gerekli? Çünkü bir dedektörün 20 480 ham sayı yerine 18 anlamlı sayıyla çalışması hem çok daha hızlı hem de çok daha anlaşılabilir; alarm çaldığında “hangi sayı yükseldi?” diye bakıp nedenini görebiliriz.

Her kayıt ve kanal 18 özneliğe indirgenir (Kod 2): 8 zaman alanı istatistiği, 6 görel bant enerjisi, spektral ağırlık merkezi ve 3 zarf enerjisi. Test 1’de rulman başına iki kanal olduğundan test 1 rulmanları 36, test 2 ve test 3 rulmanları 18 öznelikle tanımlanır.

Aşağıdaki kod, frekansla ilgili 10 özneliği hesaplar: enerjinin altı ses aralığına nasıl dağıldığı, enerjinin “ortalama tonu” ve üç hasar ritminin zarf içindeki payı.

Kod 2: Spektral ve zarf öznelikleri (src/bearing_efw/features.py’den alıntı).

```

1 BANDS = [(0, 1000), (1000, 2000), (2000, 4000), (4000, 6000), (6000, 8000), (8000, 10240)]
2 ENVELOPE_BAND = (2000, 8000)
3 FAULT_TOL_HZ = 3.0 # half-width of the window around each defect frequency
4 N_HARMONICS = 3
5 _SOS = signal.butter(4, ENVELOPE_BAND, btype="bandpass", fs=FS, output="sos")
6
7 def spectral_features(x: np.ndarray) -> dict[str, float]:
8     x = x - x.mean()
9     freqs = np.fft.rfftfreq(len(x), 1 / FS)
10    psd = np.abs(np.fft.rfft(x * np.hanning(len(x)))) ** 2 / len(x)
11    total = psd.sum()
12    out = {f"band_{lo // 1000}_{hi // 1000}k": _band_energy(freqs, psd, lo, hi) / total
13           for lo, hi in BANDS}
14    out["spectral_centroid"] = float((freqs * psd).sum() / total)
15
16    # Envelope spectrum: impacts from a localized defect modulate the resonance
17    # at the defect frequency, which shows up after demodulation.
18    env = np.abs(signal.hilbert(signal.sosfiltfilt(_SOS, x)))
19    env -= env.mean()
20    env_spec = np.abs(np.fft.rfft(env * np.hanning(len(env)))) / len(env)
21    env_total = env_spec[(freqs > 5) & (freqs < 1000)].sum()
22    for name in ("bpfo", "bpfi", "bsf"):
23        e = sum(_band_energy(freqs, env_spec,
24                             k * _FAULT_F[name] - FAULT_TOL_HZ, k * _FAULT_F[name] +
25                             FAULT_TOL_HZ)
26              for k in range(1, N_HARMONICS + 1))
27    out[f"env_{name}"] = e / env_total
28    return out

```

Kod ne yapıyor?

- **Satır 1–5 (ayarlar):** altı frekans bandının sınırları (Hz), zarf analizi için kullanılacak 2–8 kHz aralığı, hasar frekansının çevresinde bakılacak ± 3 Hz’lik pay, kaç harmoniğe (1., 2. ve 3. kata) bakılacağı ve 2–8 kHz bant geçiren filtrenin önceden hazırlanması.
- **Satır 8:** sinyalin ortalaması çıkarılır; böylece sensörün sabit kayması sonucu etkilemez.
- **Satır 9–10:** FFT ile sinyal “hangi frekansta ne kadar enerji var” biçimine çevrilir (güç

spektrumu). Hanning penceresi, kaydın başı ve sonundaki ani kesilmenin sahte frekanslar üretmesini önleyen yumuşak bir kenar inceltmesidir.

- **Satır 11–14:** her banttaki enerji toplam enerjiye bölünerek *pay* olarak saklanır; spektral ağırlık merkezi enerjinin ağırlıklı ortalama frekansıdır.
- **Satır 18:** zarf analizinin ilk iki adımı tek satırda: önce 2–8 kHz filtresi (`sosfiltfilt`; iki yönlü uygulandığı için sinyali zamanda kaydırmaz), sonra Hilbert dönüşümü ve mutlak değer (zarfın dış hattı).
- **Satır 19–21:** zarf ortalaması çıkarılır, zarfın spektrumu hesaplanır ve 5–1000 Hz arasındaki toplam zarf enerjisi bulunur.
- **Satır 22–26:** BPFO, BPFI ve BSF için, frekansın 1., 2. ve 3. katlarının ± 3 Hz çevresindeki enerji toplanır ve toplam zarf enerjisine bölünür. Sonuç “zarf enerjisinin ne kadarı bu hasar ritminde?” sorusunun cevabıdır.

Bant ve zarf enerjilerinin mutlak değil görelî olarak kullanılmasının nedeni, bu öznelîliklerin spektrumun *biçimini* tanımlaması ve RMS’in birer kopyası olmamasıdır. (Yani titreşim sadece yükseldiğinde bu sayılar değişmez; ancak enerjinin dağılımı değiştiğinde değişir. Böylece “daha gürültülü” ile “farklı türde gürültü” birbirinden ayrılır.) Öznelîlik çıkarımı bir süreç havuzuyla dosyalar üzerinde paralelleştirilmiştir (Kod 3); sonuç, her test için bir sıkıştırılmış CSV dosyasıdır.

Binlerce dosyayı tek tek işlemek yavaş olacağından, aşağıdaki kod işi bilgisayarın birden çok çekirdeğine dağıtır; her çekirdek bir grup dosyayı aynı anda işler.

Kod 3: Paralel öznelîlik çıkarımı (`scripts/build_features.py`’den alıntı).

```

1 def _process(args: tuple[str, Path]) -> list[dict]:
2     test, path = args
3     spec = TESTS[test]
4     data = load_snapshot(path)           # (20480, n_channels) float32
5     ts = parse_timestamp(path.name)     # file name = acquisition time
6     rows = []
7     for bearing, cols in spec.channels.items():
8         for axis, col in enumerate(cols):
9             rows.append({"timestamp": ts, "bearing": bearing, "channel": axis,
10                          **snapshot_features(data[:, col])})
11     return rows
12
13 with ProcessPoolExecutor(workers) as ex:
14     rows = [r for chunk in ex.map(_process, [(test, f) for f in files], chunksize=16)
15             for r in chunk]
```

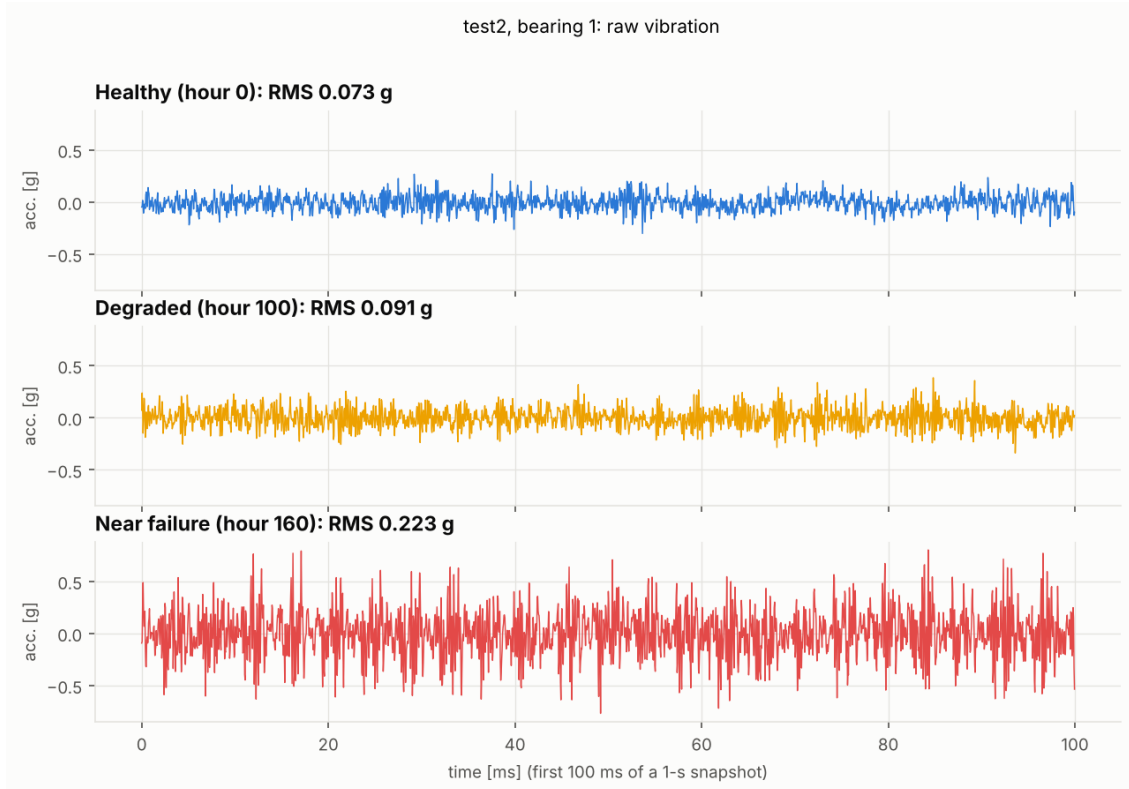
Kod ne yapıyor? `_process` fonksiyonu tek bir dosyayı işler. Satır 4 dosyayı okur (20 480 satır, her sensör için bir sütun). Satır 5 kaydın ne zaman alındığını dosya adından çıkarır; bu, sonuçları “saat” cinsinden ifade edebilmemiz için hayati önemdedir. Satır 7–10 her rulman ve o rulmanın her sensörü için 18 öznelîliği hesaplar ve zaman, rulman numarası ve sensör numarasıyla birlikte bir tablo satırı olarak saklar. Satır 13–15 bu fonksiyonu tüm dosyalara, bir işçi havuzu (`ProcessPoolExecutor`) aracılığıyla, 16’şar dosyalık paketler hâlinde paralel olarak uygular ve tüm satırları tek bir listede toplar.

5.2 Sinyal düzeyinde örnekleme

Basitçe

Bu alt bölümde, sayılara indirmeden önce titreşimin kendisine üç farklı anda bakıyoruz: rulman sağlıklıyken, bozulmaya başladığında ve kırılmak üzereyken. Amaç, öznelitliklerin neyi yakaladığını gözle görmek ve özellikle zarf analizinin hasarın yerini nasıl ortaya çıkardığını göstermektir.

Şekil 2–4, test 2’de arızalanan rulmanda öznelitliklerin neyi yakaladığını üç anda gösterir: sağlıklı (0. saat), bozulmuş (100. saat, alarmdan sonra) ve arızaya yakın (160. saat). Grafik etiketleri depodaki grafiklerle aynı kalması için İngilizcedir (“Healthy” = sağlıklı, “Degraded” = bozulmuş, “Near failure” = arızaya yakın).



Şekil 2: Test 2, rulman 1’in ham titreşimi (üç kaydın ilk 100 ms’si).

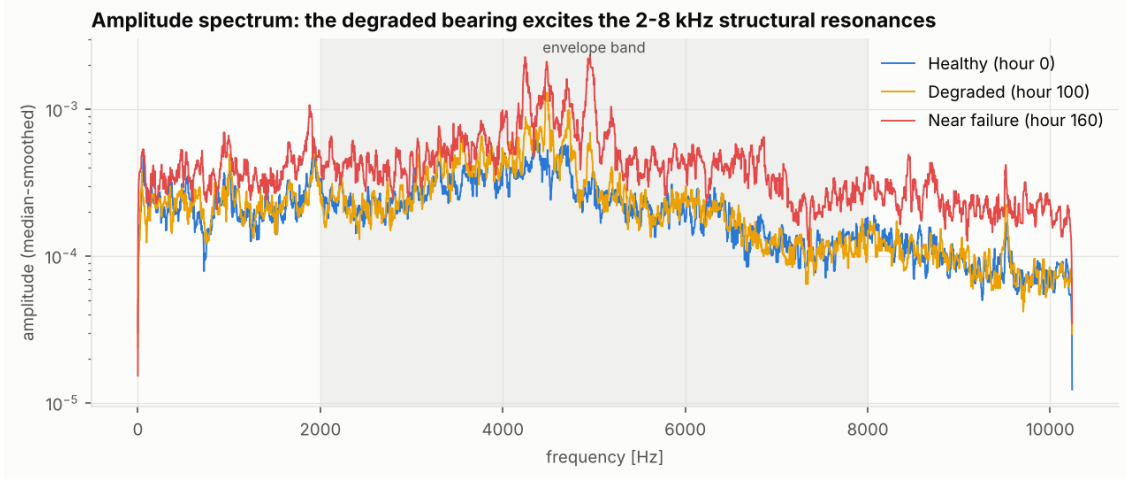
Bu grafik nasıl okunur?

Üst üste üç küçük grafik var; her biri aynı rulmanın farklı bir andaki kaydını gösterir: mavi sağlıklı (0. saat), turuncu bozulmuş (100. saat), kırmızı arızaya yakın (160. saat). Her grafiğin başlığında o kaydın RMS değeri (ortalama titreşim gücü) yazar. **Yatay eksen** zamandır, milisaniye (ms, saniyenin binde biri) cinsinden; bir saniyelik kaydın yalnızca ilk 100 ms’si (saniyenin onda biri) çizilmiştir. **Dikey eksen** ivmedir (“acc.”), **g** biriminde (1 g, yerçekiminin yarattığı ivmeye eşittir); sensörün o anda ne kadar sarsıldığını gösterir. Üç grafiğin dikey eksenini aynıdır, böylece genlikler doğrudan karşılaştırılabilir. Çizgi ne kadar yukarı-aşağı geniş salınıyorsa titreşim o kadar güçlüdür; tek tük sivri iğneler darbelerle işaret eder.

Ne görüyoruz? 100. saatte dalga biçimi görsel olarak sağlıklı olana hâlâ benzer ve RMS yalnızca 0,073’ten 0,091 g’ye (+%25) çıkmıştır. Arızaya yakın dönemde genlik üç katına çıkmış (0,223 g) ve sinyal belirgin biçimde darbeli hâle gelmiştir; kırmızı grafikte sık sık sivri iğneler görülür.

Ne anlama geliyor? Bozulma başladığında (100. saat) bu artış gözle veya sabit bir titreşim sınırıyla kolayca gözden kaçabilir. Ancak rulman kırılmak üzereyken fark gözle bile bellidir; o noktada ise uyarı için artık geç kalınmıştır.

Neden önemli? Dolayısıyla erken uyarı, dalga biçimine bakmaktan veya toplam seviyeden fazlasına dayanmalıdır. Bu, sonraki iki şekildeki daha ince analizlerin gerekçesidir.



Şekil 3: Aynı üç kaydın genlik spektrumu (medyan filtreye yumuşatılmış, logaritmik ölçek).

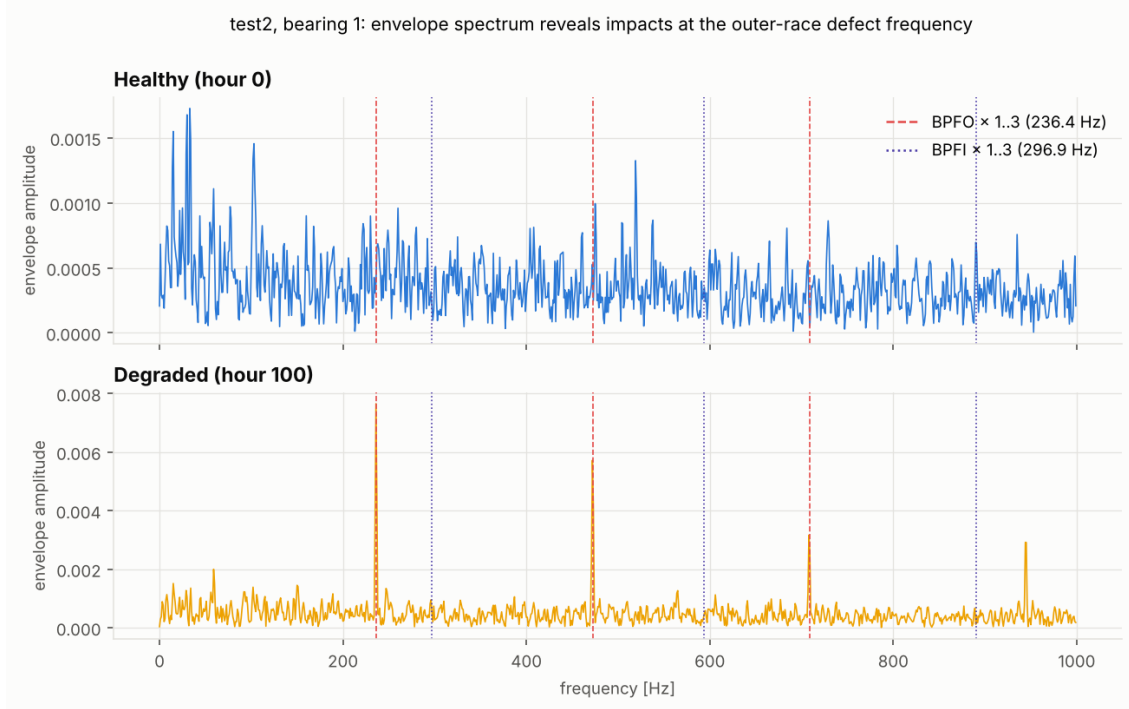
Bu grafik nasıl okunur?

Bu bir *spektrum*dur: titreşimin hangi frekansta (tonda) ne kadar güçlü olduğunu gösterir; bir ekolayzır ekranındaki çubuklar gibi. **Yatay eksen** frekanstır, Hz (saniyedeki salınım sayısı) cinsinden, 0'dan yaklaşık 10 000 Hz'e (sola doğru bas, sağa doğru tiz tonlar). **Dikey eksen** o frekanstaki genliktir ("amplitude"); çizgileri okunur kılmak için kısa aralıklarla medyan alınarak yumuşatılmıştır. Dikey eksen **logaritmiktir**: her büyük çizgi bir öncekinin 10 katıdır (10^{-5} , 10^{-4} , 10^{-3}); yani grafikte küçük görünen bir yükselme gerçekte birkaç kat artış olabilir. Renkler önceki şekildeki gibidir: mavi sağlıklı, turuncu bozulmuş, kırmızı arızaya yakın. Gri gölgeli alan ("envelope band") zarf analizinde kullanılan 2–8 kHz aralığıdır.

Ne görüyoruz? Üç spektrumun da enerjisi yaklaşık 3 ile 6 kHz arasında toplanır. Rulman bozuldukça bu bölgedeki tepeler büyür ve genişler; arızaya yakın dönemde (kırmızı) yüksek frekans tabanı (6–10 kHz) neredeyse bir merteye (yaklaşık 10 kat) yükselir.

Ne anlama geliyor? 3–6 kHz'deki tepeler rulman gövdesinin yapısal rezonanslarıdır: gövdenin doğal "çınlama" tonları. Hasar darbeleri bu tonları daha güçlü çınlatır. Bu durum hem görelî bant enerjisi özniteliklerini hem de zarf analizi için 2–8 kHz bandının seçimini doğrular.

Neden önemli? Ancak spektrum, *hangi* parçanın hasarlı olduğunu göstermez: "bir şeyler daha gürültülü" der ama "nerede?" sorusunu cevaplamaz. Bunun için bir sonraki şekildeki zarf spektrumu gerekir.



Şekil 4: Sağlıklı ve bozulmuş kayıtların zarf spektrumu; BPFO’nun (kesikli) ve BPFI’nin (noktalı) ilk üç harmoniği işaretlenmiştir.

Bu grafik nasıl okunur?

İki grafik var: üstte sağlıklı (mavi, 0. saat), altta bozulmuş (turuncu, 100. saat) kayıt. Her biri bir *zarf spektrumu*dur; yani “darbeler hangi ritimle tekrarlanıyor?” sorusunun cevabı. **Yatay eksen** bu tekrarlanma ritmidir, Hz cinsinden (0–1000 Hz; ör. 236 Hz “saniyede 236 kez” demektir). **Dikey eksen** zarf genliğidir (“envelope amplitude”): o ritmin ne kadar güçlü olduğu; ölçek doğrusaldır. *Dikkat:* iki grafiğin dikey ölçeği farklıdır; üstte en fazla yaklaşık 0,0017, altta 0,008. **Kırmızı kesikli dikey çizgiler** dış bilezik hasar ritminin (BPFO = 236,4 Hz) 1, 2 ve 3 katlarını, **mor noktalı dikey çizgiler** iç bilezik hasar ritminin (BPFI = 296,9 Hz) 1, 2 ve 3 katlarını işaretler. Bir çizginin üstünde keskin bir tepe varsa, o parçada hasar olduğuna dair güçlü bir işarettir.

Ne görüyoruz? Bu, teşhis açısından en önemli şekildir. Sağlıklı zarf spektrumu, arıza frekanslarında hiçbir yapı içermeyen düz bir gürültü tabanıdır (birkaç düşük frekanslı tepe olsa da hiçbiri işaretli çizgilerin üstüne düşmez). 100. saatte tam olarak 236, 473 ve 709 Hz’de, yani BPFO’nun 1 $\times$, 2 $\times$ ve 3 $\times$ katlarında, gürültü tabanının yaklaşık on katı genlikte keskin tepeler belirir; iç bilezik frekanslarında (mor noktalı çizgiler) ise hiçbir şey görünmez. (Sağ uçta yaklaşık 945 Hz’deki daha küçük tepe de BPFO’nun 4. katına denk gelir.)

Ne anlama geliyor? Bu, lokal bir **dış bilezik** hasarının ders kitabı imzasıdır: dış halkadaki bir çukurun üzerinden saniyede tam 236 kez makara geçiyor ve her geçişte bir “tık” oluşuyor. Şekil, rulman geometrisinin, arıza frekansı hesabının ve zarf hattının doğru olduğunu doğrular; *env_bpfo* öz niteliğinin ölçtüğü de budur.

Neden önemli? Bu imza testin bitiminden yaklaşık üç gün önce görünürdür. Yani sistem yalnızca “bir sorun var” demekle kalmayıp “sorun büyük ihtimalle dış bilezikte” diyebilir; bakım ekibi doğru parçayı hazırlayabilir.

5.3 Ön işleme ve veri bölmeleri

Basitçe

Bu adımda iki iş yapılır. *Birincisi*, her rulmanın sayıları kendi “normal”ine göre ölçeklenir. Neden? Bazı rulmanlar doğuştan biraz daha gürültülüdür; bir insanın normal ateşinin 36,2, bir başkasının 36,8 olması gibi. “36,8 ateş mi?” sorusunun cevabı kişiye bağlıdır. Ölçekleme sonrası her sayı “bu rulmanın normaline göre kaç adım yukarıda ya da aşağıda” anlamına gelir. *İkincisi*, her deney zamana göre üç döneme bölünür: ilk %25 *eğitim* (sistem normali öğrenir), sonraki %10 *kalibrasyon* (alarm sınırı belirlenir), geri kalanı *test* (sistem gerçekten sınanır). Bu sıra hiç bozulmaz: sistem asla gelecekteki veriyle eğitilmez.

Öznitelikler her rulman için bir matrise (satır ve sütunlardan oluşan sayı tablosuna) dönüştürülür (her satır bir kayıt, zaman sırasıyla). Kesin pozitif ve ağır kuyruklu (arada bir çok büyük değerler alabilen) özniteliklerin logaritması alınır; ardından her öznitelik, o rulmanın kendi eğitim döneminin ortalaması ve standart sapmasıyla standartlaştırılır (Kod 4). Bu adımdan sonra +3 değeri, rulman doğası gereği gürültülü ya da sessiz olsun, “bu rulmanın normalinin üç sağlıklı standart sapma üstü” anlamına gelir; böylece bir testteki tüm rulmanlar tek bir dedektörle izlenebilir.

Aşağıdaki kod, her rulman için hem dönemleri belirler hem de logaritma ve ölçekleme işlemini yapar.

Kod 4: Zaman sıralı bölmeler ve rulman bazında standartlaştırma (`src/bearing_efw/preprocess.py`’den alıntı).

```
1 for bearing, w in wide_by_bearing(df).items():
2     hours = ((w.index - t0).total_seconds() / 3600).to_numpy()
3     # Splits use the snapshot index, not wall-clock time: test1 was recorded in
4     # bursts with multi-day gaps, so a time-based split would leave almost no
5     # training data.
6     life = np.arange(len(w)) / len(w)
7     train = life < train_end # first 25 %
8     calib = (life >= train_end) & (life < calib_end) # next 10 %
9
10    Z = w.copy()
11    log_cols = [c for c in Z.columns if c.rsplit("_ch", 1)[0] in LOG_FEATURES]
12    Z[log_cols] = np.log(Z[log_cols].clip(lower=1e-9))
13    mu, sd = Z[train].mean(), Z[train].std().replace(0, 1)
14    X = ((Z - mu) / sd).to_numpy(dtype=np.float32)
```

Kod ne yapıyor? Satır 1 her rulmanı sırayla ele alır. Satır 2 her kaydın testin başından kaç saat sonra alındığını hesaplar. Satır 6, her kayda “ömrün yüzde kaçında” olduğunu söyleyen bir sayı verir (0 = ilk kayıt, 1’e yakın = son kayıt); dikkat, bu saat değil *kayıt sırasına* göredir (satır 3–5’teki yorum nedenini açıklar: test 1’de günlerce süren kayıt boşlukları var). Satır 7–8 ilk %25’i eğitim, sonraki %10’u kalibrasyon olarak işaretler. Satır 11–12 çarpıklık dışındaki tüm özniteliklerin logaritmasını alır (sıfırın logaritması tanımsız olduğu için değerler önce çok küçük bir sayıyla alttan sınırlanır). Satır 13 *yalnızca eğitim dönemi* kullanılarak her özniteliğin ortalamasını ve standart sapmasını bulur (standart sapması sıfır çıkan bir sütun 1 ile değiştirilir ki sıfıra bölme olmasın). Satır 14 standartlaştırmayı yapar: her değerden ortalamayı çıkarır ve standart sapmaya böler; sonuç z-skorudur.

Tablo 4: Rulman başına bölme boyutları.

Test	Kayıt	Eğitim	Kalibrasyon	Eğitim bitişi [sa]	Kalibrasyon bitişi [sa]
test1	2 156	539	216	290,1	433,8
test2	982	246	98	40,8	57,2
test3	6 323	1 581	633	267,5	380,2

Tabloyu okumak için: örneğin test 2’de her rulmanın 982 kaydı var; ilk 246 kayıt (testin ilk 40,8 saati) eğitim, sonraki 98 kayıt (57,2. saate kadar) kalibrasyon için kullanılır. Geri kalan her şey sistemin hiç görmediği veridir.

5.4 Dedektörler

Basitçe

Dedektör, her kayda bir “tuhaflik puanı” veren parçadır. Dört farklı dedektör deniyoruz ve hepsinin aynı işi yapmasını sağlıyoruz: *öğren* (sağlıklı verilerden normal çıkar) ve *puanla* (her yeni kayda bir sayı ver). Böylece dördünü aynı yarışta adil biçimde karşılaştırabiliyoruz. Dört rulmanın sağlıklı verisini birlikte kullanıyoruz; bu, öğrenilecek örnek sayısını dört katına çıkarır, tıpkı bir doktorun tek hastanın değil dört sağlıklı kişinin kalp sesini dinleyerek “normal”i daha iyi öğrenmesi gibi.

Tüm dedektörler ortak bir arayüze sahiptir: `fit` dört rulmanın sağlıklı eğitim matrislerinin listesini alır, `score` ise bir rulmanın tüm matrisini kayıt başına bir puana dönüştürür. Tüm rulmanların standartlaştırılmış sağlıklı verisini birleştirerek eğitmek, rulman başına ayrı model eğitmeye göre eğitim kümesini dört katına çıkarır.

Aşağıdaki kod, üç klasik dedektörün tamamıdır; her biri yalnızca birkaç satırdır, çünkü ağır işi hazır kütüphaneler yapar.

Kod 5: Klasik dedektörler (`src/bearing_efw/models.py`’den alıntı).

```

1 class RMSThreshold:
2     """Classic condition-monitoring baseline: overall vibration level (z-score of log RMS)."""
3     def fit(self, Xs): return self
4     def score(self, X): return X[:, self.idx].max(axis=1) # idx = RMS columns
5
6 class Mahalanobis:
7     """Distance from the healthy feature distribution (shrinkage covariance)."""
8     def fit(self, Xs):
9         self.cov = LedoitWolf().fit(np.vstack(Xs))
10        return self
11    def score(self, X):
12        return np.sqrt(self.cov.mahalanobis(X))
13
14 class IForest:
15     def __init__(self, n_estimators=300, seed=0):
16         self.model = IsolationForest(n_estimators=n_estimators, random_state=seed)
17     def fit(self, Xs):
18         self.model.fit(np.vstack(Xs))
19         return self
20     def score(self, X):
21         return -self.model.score_samples(X) # higher = more anomalous

```

Kod ne yapıyor?

- **RMS eşiği (satır 1–4):** öğrenecek bir şeyi yoktur (satır 3), çünkü veriler zaten rulmanın kendi normaline göre ölçeklenmiştir. Puanı, RMS sütunlarının en büyüğüdür (satır 4; test 1’de her rulmanın iki sensörü olduğu için iki RMS değerinden büyük olanı alınır). Yani “titreşim gücü normalin kaç adım üstünde?”
- **Mahalanobis (satır 6–12):** öğrenirken dört rulmanın sağlıklı verisini üst üste koyar (`np.vstack`) ve Ledoit–Wolf yöntemiyle ilişki tablosunu (kovaryansı) hesaplar (satır 9). Puanlarken her kaydın bu sağlıklı buluttan uzaklığını verir (satır 12).
- **Isolation Forest (satır 14–21):** 300 rastgele ağaçtan oluşan bir orman kurar; sabit tohum (`seed=0`) her çalıştırmada aynı “rastgele” ağaçların kurulmasını sağlar (satır 16). Kütüphane

“ne kadar normal” puanı verdiği için satır 21 işareti ters çevirir; böylece büyük puan her dedektörde “daha tuhaf” anlamına gelir.

LSTM otokodlayıcı mimarisi. Kodlayıcı LSTM (32 gizli birim) ardışık $W = 12$ kayıttan (yaklaşık iki saat) oluşan bir pencereyi okur; son gizli durumu 8 boyutlu bir gizli koda (8 sayılı bir özete) izdüşürülür. Kod çözücü bu kodu 12 adım boyunca tekrarlar, ikinci bir LSTM (32 birim) ve doğrusal bir katmandan geçirerek öznitelik boyutuna geri döndürür. Model 18 öznitelik için 16 250 (test 1’in 36 özniteliği için 19 148) parametreye (eğitim sırasında ayarlanan iç sayıya) sahiptir. Model Adam ile (öğrenme oranı 2×10^{-3} , yığın boyutu 64; yani her adımda 64 örneğe bakarak küçük düzeltmeler yapan standart bir eğitim yöntemiyle) 60 epok (eğitim verisinin baştan sona 60 kez gözden geçirilmesi) boyunca *gürültü giderici* (denoising) otokodlayıcı olarak eğitilir: girdiye standart sapması 0,1 olan Gauss gürültüsü eklenir ve modelden temiz pencereyi yeniden oluşturması istenir. Yalnızca yaklaşık 1 000 pencerelik bir eğitim kümesinde (test 2) bu, ağırların eğitim verisini ezberlemesini önler. Pencereler rulman bazında oluşturulur, hiçbir zaman iki rulmanı kapsamaz ve t kaydının penceresi t ’de biter; dolayısıyla puan yalnızca geçmiş veriyi kullanır ve çevrimiçi (online; veri geldikçe, gerçek zamanlı) çalışabilir.

Basitçe

LSTM otokodlayıcı, son iki saatlik (12 kayıt) gidişatı okuyup 8 sayılı kısa bir özete sıkıştıran, sonra bu özetten iki saatlik gidişatı yeniden kurmaya çalışan küçük bir yapay sinir ağıdır. “LSTM” kısmı, ağırların sırayla gelen verilerde neyin önce neyin sonra olduğunu hatırlayabilmesini sağlar. Eğitim sırasında girdiye bilerek biraz “parazit” eklenir ve ağırların temiz hâlini çıkarması istenir; bu, öğrenciye aynı soruyu her seferinde biraz farklı sorup ezberlemesini engellemek gibidir. Ağ yalnızca geçmişe bakar; geleceği hiçbir zaman görmez, bu yüzden gerçek bir makinede canlı olarak çalıştırılabilir.

Aşağıdaki kod ağı yapısını, nasıl eğitildiğini ve nasıl puan verdiğini gösterir.

Kod 6: LSTM otokodlayıcı (src/bearing_efw/models.py’den alıntı).

```

1 class Net(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.enc = nn.LSTM(n_features, hidden, batch_first=True)
5         self.to_latent = nn.Linear(hidden, latent)
6         self.from_latent = nn.Linear(latent, hidden)
7         self.dec = nn.LSTM(hidden, hidden, batch_first=True)
8         self.out = nn.Linear(hidden, n_features)
9
10    def forward(self, x):
11        # x: (batch, window, features)
12        _, (h, _) = self.enc(x)
13        z = self.to_latent(h[-1])
14        # latent code
15        rep = self.from_latent(z).unsqueeze(1).repeat(1, x.shape[1], 1)
16        y, _ = self.dec(rep)
17        return self.out(y)
18
19    def fit(self, Xs):
20        # Windows are built per bearing so a sequence never spans two bearings.
21        W = torch.tensor(np.concatenate([self._windows(X)[self.window - 1:] for X in Xs]))
22        opt = torch.optim.Adam(self.net.parameters(), lr=self.lr)
23        for _ in range(self.epochs):
24            for idx in np.array_split(self.rng.permutation(len(W)), max(1, len(W) //
25                self.batch_size)):
26                xb = W[idx]
27                # denoising objective: reconstruct the clean window from a noisy copy
28                loss = ((self.net(xb + self.noise * torch.randn_like(xb)) - xb) ** 2).mean()
29                opt.zero_grad(); loss.backward(); opt.step()

```

```

28 def score(self, X):
29     with torch.no_grad():
30         W = torch.tensor(self._windows(X))          # window ending at each snapshot
31         return ((self.net(W) - W) ** 2).mean(dim=(1, 2)).numpy()

```

Kod ne yapıyor?

- **Satır 1–8 (ağın parçaları):** bir kodlayıcı LSTM (okur), sıkıştırma katmanı (32 sayıyı 8 sayıya indirir), açma katmanı (8’i tekrar 32’ye çıkarır), bir kod çözücü LSTM (zaman içinde yeniden kurar) ve öznitelik sayısına geri dönen son katman.
- **Satır 10–15 (bir pencere ağdan nasıl geçer):** kodlayıcı pencereyi okur ve son “hafıza” durumunu verir (satır 11); bu, 8 sayılık özete sıkıştırılır (satır 12); özet pencere uzunluğu kadar (12 kez) çoğaltılır (satır 13); kod çözücü bunu zamana yayar (satır 14) ve son katman her an için öznitelikleri yeniden üretir (satır 15).
- **Satır 17–26 (eğitim):** her rulmanın sağlıklı verisinden 12’lik pencereler kesilir ve birleştirilir (satır 19; pencereler iki rulmanı asla karıştırmaz). 60 tur boyunca (satır 21) pencereler karıştırılıp 64’lük gruplara bölünür (satır 22). Her grupta girdiye küçük rastgele gürültü eklenir, ağın çıktısı *temiz* pencereyle karşılaştırılır ve farkın karelerinin ortalaması hata olarak hesaplanır (satır 25). Satır 26 bu hatayı biraz azaltacak yönde ağın iç sayılarını günceller.
- **Satır 28–31 (puanlama):** her kayıt için o kayıta *biten* pencere oluşturulur (satır 30; yani yalnızca geçmiş kullanılır), ağdan geçirilir ve gerçek ile yeniden üretilen arasındaki ortalama kare fark puan olarak döndürülür (satır 31). Bu, yukarıdaki e_t formülünün ta kendisidir. `no_grad` “burada öğrenme yok, sadece hesapla” demektir.

5.5 Puandan alarma

Basitçe

Bir puan tek başına bir şey söylemez; “ne zaman alarm çalsın?” kuralına ihtiyaç var. Üç parçalı bir kural kullanıyoruz. (1) *Eşik*: kalibrasyon döneminde puanların normalde ne kadar yükseldiğine bakıp, bunun biraz üstüne bir sınır çiziyoruz. (2) *Debounce (sakinleştirme)*: tek bir yüksek puan alarm çaldırmaz; son 6 kaydın (yaklaşık bir saat) en az 4’ü sınırı aşmalıdır. Bu, bir duman dedektörünün tost makinesinden çıkan tek bir duman bulutunda değil, duman sürdüğünde ötmesi gibidir. (3) *Kalıcılık*: gerçek bir hasar kendiliğinden iyileşmez; bu yüzden uyarı süresini hesaplarken, test sonuna kadar hiç susmayan alarmın başladığı anı esas alıyoruz.

Eşik. Her rulman ve dedektör için eşik yalnızca kalibrasyon puanlarından (s) türetilir:

$$\tau = Q_{0.995}(s) + m (Q_{0.995}(s) - \text{medyan}(s)), \quad m = 0,5. \quad (8)$$

Formüldeki simgeler. s , kalibrasyon dönemindeki tüm puanlardır. τ (tau) bulunan alarm eşiğidir. $Q_{0.995}(s)$, puanların %99,5’inin altında kaldığı değerdir (0,995 kuantili); kabaca “sağlıklı dönemin neredeyse en yüksek puanı”. $\text{medyan}(s)$, puanların tam ortasındaki değer, yani tipik sağlıklı puandır. m ek pay çarpanıdır (0,5).

Sezgisel anlamı. Önce sağlıklı dönemin neredeyse en yüksek puanını alıyoruz. Sonra buna, “tipik puan ile neredeyse en yüksek puan arasındaki mesafenin yarısı” kadar ek bir pay ekliyoruz. Böylece eşik, sağlıklı makinenin olağan dalgalanmasının rahatça üstünde kalır. Bir benzetme: bir yoldaki araçların neredeyse hepsi 95 km/sa’in altında gidiyorsa ve tipik hız 80 km/sa ise, “şüpheli hız” sınırını 95’in biraz üstüne, $95 + 0,5 \times (95 - 80) = 102,5$ km/sa’e koyarız.

Pay (margin), sağlıklı puanların yayılımına göre ifade edildiğinden kural her ölçek ve işaretteki puan için çalışır (RMS z-puanı negatif olabilir, diğer puanlar olamaz).

Debounce. Eşiği aşan tek bir gürültülü kayıt operatörü uyarmamalıdır. Son $n = 6$ kaydın (bir saat) en az $k = 4$ 'ü τ 'yu aştığında t anında alarm etkindir. (Buna “k-of-n” yani “n’nin k’sı” kuralı denir.)

Kalıcı alarm. Gerçek bir bozulma kendiliğinden düzelmez. Arızalı bir rulman için raporlanan uyarı, kaydın sonuna kadar sürmesi şartıyla *son* alarm bölümünün (epizodunun; alarmin kesintisiz açık kaldığı bir zaman diliminin) başlangıcıdır; daha önce başlayıp sona eren bölümler ayrıca sayılır.

Aşağıdaki kod bu üç kuralı uygular; her biri birkaç satırdır.

Kod 7: Eşik, k-of-n debounce ve kalıcı alarm (src/bearing_efw/alarm.py).

```

1 def fit_threshold(calib_scores, quantile, margin):
2     q, med = np.quantile(calib_scores, quantile), np.median(calib_scores)
3     return float(q + margin * (q - med))
4
5 def k_of_n(exceed: np.ndarray, k: int, n: int) -> np.ndarray:
6     """Alarm at t when at least k of the last n snapshots (inclusive) exceed the threshold."""
7     c = np.convolve(exceed.astype(int), np.ones(n, dtype=int))[: len(exceed)]
8     return c >= k
9
10 def first_persistent_alarm(alarm, hours, start_h):
11     """Start of the final alarm episode, if it lasts until the end of the record."""
12     eps = [(s, e) for s, e in alarm_episodes(alarm) if hours[s] >= start_h]
13     if not eps:
14         return None
15     return float(hours[eps[-1][0]]) if eps[-1][1] == len(alarm) else None

```

Kod ne yapıyor?

- **Satır 1–3 (eşik):** kalibrasyon puanlarının 0,995 kuantilini ve medyanını hesaplar, sonra yukarıdaki formülü birebir uygular.
- **Satır 5–8 (debounce):** her kayıt için “eşik aşıldı mı?” bilgisini 1 (evet) ya da 0 (hayır) olarak alır. Satır 7, her an için son 6 kayıttaki 1’lerin sayısını hesaplar (6 kayıtlık kayan bir pencerede toplama; `convolve` bunu tek seferde yapar). Satır 8, bu sayı 4 veya daha fazlaysa “alarm açık” der.
- **Satır 10–15 (kalıcı alarm):** alarmin açık kaldığı kesintisiz dönemleri (epizotları) bulur ve yalnızca kalibrasyon bittikten sonra başlayanları tutar (satır 12). Hiç yoksa “kalıcı alarm yok” döner (satır 13–14). Varsa, *son* epizot kaydın sonuna kadar sürüyorsa onun başlangıç saatini, sürmüyorsa “kalıcı alarm yok” döndürür (satır 15).

5.6 Teşhis

Basitçe

Alarm çaldığında sistem “nerede?” sorusuna cevap arar. Alarm veren rulmanın zarf ölçümlerine (dış bilezik, iç bilezik ve makara ritimleri) sonraki 12 kayıt (yaklaşık iki saat) boyunca bakar. Hangi ritim normaline göre en çok yükselmişse onu söyler; ama sadece yükselme gerçekten belirginse (normalin 3 standart sapmasından fazla). Değilse tahmin yürütmek yerine dürüstçe “belirgin bir hasar ritmi yok” der. Bu, emin olmayan bir doktorun “şu olabilir” yerine “tahlil net değil” demesi gibidir.

Kalıcı bir alarm verildiğinde sistem, alarm veren rulmanın standartlaştırılmış zarf özniteliklerine sonraki 12 kayıt boyunca bakar ve ortalama z-puanı en yüksek olan arıza frekansını, 3'ü aşıyorsa, raporlar (Kod 8). Aksi hâlde tahmin yürütmek yerine “belirgin arıza frekansı yok” der.

Kod 8: Zarfa dayalı teşhis (scripts/run_experiment.py’den alıntı).

```

1 DEFECTS = {"env_bpfo": "outer race", "env_bpfi": "inner race", "env_bsfi": "rolling element"}
2
3 def diagnose(d, alarm_h, n=12):
4     if alarm_h is None:
5         return ""
6     cols = list(d.raw.columns)
7     after = np.flatnonzero(d.hours >= alarm_h)[:n]
8     z = {k: np.mean([d.X[after, cols.index(c)].mean() for c in cols if c.startswith(k + "_")])
9           for k in DEFECTS}
10    best = max(z, key=z.get)
11    return f"{DEFECTS[best]} (z={z[best]:.1f})" if z[best] > 3 else "no clear defect frequency"

```

Kod ne yapıyor? Satır 1 her zarf öznitelikliğini bir parça adıyla eşleştirir (“outer race” = dış bilezik, “inner race” = iç bilezik, “rolling element” = yuvarlanma elemanı). Satır 4–5: kalıcı alarm yoksa teşhis de yoktur. Satır 7 alarmın başladığı andan itibaren ilk 12 kaydı seçer. Satır 8–9 her parça için ilgili zarf sütunlarının (test 1’de iki sensör olduğundan iki sütun olabilir) bu 12 kayıttaki ortalama z-skorumu hesaplar. Satır 10 en yüksek olanı seçer. Satır 11: bu değer 3’ten büyükse parçanın adını ve z-değerini, değilse “belirgin arıza frekansı yok” yazar.

5.7 Değerlendirme protokolü

Basitçe

Sistemi nasıl notladığımızı anlatır. Zorluk şu: veri, rulmanın *test sonunda* bozuk olduğunu söylüyor ama bozulmanın *ne zaman başladığını* söylemiyor. Bu yüzden iki not veriyoruz. (1) *Uyarı süresi*: kalıcı alarm, test bitmeden kaç saat önce başladı? Ne kadar uzunsa o kadar iyi; bakım ekibine o kadar hazırlık zamanı kalır. (2) *Yanlış alarm*: makinenin hâlâ sağlıklı olduğundan emin olduğumuz dönemde kaç kez boşuna alarm çaldı? Ne kadar azsa o kadar iyi; sık sık boşuna öten bir alarm kimse güvenmez. Ayrıca bütün ayarları yalnızca bir deney (test 2) üzerinde yaptık; diğer iki deneyi hiç görmeden sona sakladık. Bu, öğrencinin çalışırken görmediği sorulardan oluşan bir sınav gibidir ve sonuçların dürüst olmasını sağlar.

Veri seti yalnızca her test durdurulduğunda bir rulmanın arızalı olduğunu belirtir; bozulmanın başlangıcı etiketli değildir. Buradan iki metrik çıkar:

- Arızalı bir rulmanın **uyarı süresi**: kalıcı alarm ile testin bitiş arasındaki saat (test süresinin yüzdesi olarak da verilir). Yüksek olması daha iyidir.
- **Yanlış alarmlar**: sağlıklı olduğuna inanılan bir pencere içinde *herhangi bir* rulmanda verilen alarm bölümleri. Bu pencere kalibrasyondan sonra (kayıtların %35’i) başlar ve test 1, 2 ve 3 için sırasıyla kayıtların %60, %50 ve %85’inde biter. Bu sınırlar durum göstergelerinin incelenmesinden gelir (Bölüm 6) ve yapılandırma dosyasında belgelenmiştir. Oran, bu penceredeki 1000 rulman-saat başına normalleştirilir (bir rulmanın bir saat izlenmesi 1 rulman-saattir; dört rulmanı 10 saat izlemek 40 rulman-saat eder).

Hiperparametreler (yöntemin elle seçilen ayarları: bölme oranları, eşik yüzdeliği ve payı, k ve n , ağ boyutu, epok sayısı, gürültü) yalnızca test 2 üzerinde seçilmiştir. Test 1 ve test 3 hiçbir karar için kullanılmamış ve ayrılmış değerlendirme kümeleri olarak kalmıştır.

5.8 Tasarım kararları

Basitçe

Bu liste, sistemi kurarken bilerek verdiğimiz ve sonuçların güvenilir olmasını sağlayan kararlardır. Özetle: geleceği asla geçmişe karıştırmak; her kaydı zamandaki bir an olarak tut; her rulmanı ayrı izle; dönemleri saate değil kayıt sayısına göre böl (çünkü kayıtlarda boşluklar var); her rulmanı kendi normaliyiyle kıyasla; bütün yöntemlere aynı kuralları uygula; ve her çalıştırmada aynı sonucu al.

- **Baştan eğit, gerisinde test et.** Tüm bölmeler zaman sıralıdır. Tüm deneyden alınan pencereleri karıştırmak arızayı eğitime sızdırır.
- **Bir kayıt, zamanda bir noktadır.** Öznitelikler dosya başına hesaplanır, böylece sonuçlar “arızadan kaç saat önce” cinsinden ifade edilir.
- **Doğru rulmanı ve her rulmanı izle.** Her rulman ayrı puanlanır; arızalanan rulmanlar veri setinin belgelerinden bilinir ve yalnızca değerlendirmede kullanılır.
- **Saate göre değil, kayıt sayısına göre böl.** Test 1’deki çok günlük boşluklar nedeniyle *sürenin* ilk %25’i çok az kayıt içerirdi.
- **Rulman bazında temel seviye.** Her rulmanın kendine göre standartlaştırılması, doğal titreşim seviyeleri farklı rulmanların tek bir modelle izlenmesini sağlar.
- **Tüm dedektörler için aynı eşik kuralı ve alarm mantığı;** böylece sonuçlardaki farklar ayar farkından değil dedektörlerin kendisinden gelir.
- **Belirlenimcilik (determinism).** Tüm rastgele bileşenler sabit tohumla (rastgele sayı üreticinin her seferinde aynı başlangıç noktasıyla) başlatılır; art arda iki çalıştırma aynı sonuç tablolarını üretir.

6 Sonuçlar

6.1 Ana sonuçlar

Basitçe

Bu bölüm, dört yöntemin “sınav notlarını” verir. Aşağıdaki ana tabloda her satır bir yöntem, ilk dört sayı sütunu bozulan dört rulmandan biri, son sütun ise yanlış alarm sıklığıdır. Tabloyu okumadan önce bir hücrenin ne anlama geldiğine bir örnekle bakalım.

Bir hücre nasıl okunur? Tablo 5’de “RMS eşiği” satırı ile “test2, R1 dış bil.” sütununun kesiştiği hücrede **74,3** yazar. Anlamı: test 2’de 1 numaralı rulman (dış bileziğinden) bozulmadan ve test durdurulmadan **74,3 saat önce**, yani yaklaşık **3 gün önce**, RMS eşiği yöntemi kalıcı bir alarm verdi ve bu alarm test bitene kadar hiç susmadı. Sayı ne kadar büyükse uyarı o kadar erkendir. Aynı satırda “test1, R4 yuv. el.” sütunundaki **176,6** ise test 1’de 4 numaralı rulmanın (makara hasarı) yaklaşık 7 gün önceden fark edildiği anlamına gelir. Kısaltmalar: “R3” = rulman 3, “iç bil.” = iç bilezik, “yuv. el.” = yuvarlanma elemanı (makara), “dış bil.” = dış bilezik.

Yanlış alarm sütunu nasıl okunur? Son sütun, makinenin sağlıklı olduğuna inandığımız dönemlerde kaç kez boşuna alarm çaldığını, **1 000 rulman-saat başına** verir. “Rulman-saat”, izlenen rulman sayısı ile izleme süresinin çarpımıdır. Örneğin RMS eşiği için **5,34**: tek bir rulmanı 1 000 saat (yaklaşık 42 gün) izleseydik ortalama 5 kez kadar boşuna alarm görürdük. Dört rulmanlı

bir makineyi izlediğimizi düşünürsek 1 000 rulman-saat, 4 rulman $\times$ 250 saat, yani yaklaşık 10 gündür; bu da kabaca 10 günde 5 kez, yani **yaklaşık iki günde bir** boşuna alarm demektir. LSTM otokodlayıcının **12,67**'si bunun iki katından fazlasıdır: aynı makinede **günde birden biraz fazla** boşuna alarm; Mahalanobis'in **0,67**'si 1 000 rulman-saatte bir yanlış alarmdan bile azdır; Isolation Forest'ın **0,00**'i ise bu dönemlerde hiç yanlış alarm vermediği anlamına gelir. Burada sayı ne kadar *küçükse* o kadar iyidir.

Neden ikisi birden mükemmel olamıyor? Bir duman dedektörünün hassasiyet ayarını düşünün. Çok hassas ayarlarsanız yangını en erken anda fark eder, ama her ekmek kızarttığınızda da öter; bir süre sonra pilini çıkarırsınız. Az hassas ayarlarsanız hiç boşuna ötmez, ama yangını ancak oda dumanla dolunca fark eder. Dedektörlerimiz de böyledir: küçük değişikliklere hızlı tepki veren bir yöntem hem gerçek hasarı erken yakalar hem de zararsız dalgalanmalara (ör. makine durup yeniden çalıştığında) alarm verir. Bu yüzden tabloda en erken uyarı yöntemlerin, aynı zamanda en çok yanlış alarm veren yöntemler olduğunu göreceğiz. Hangisinin “en iyi” olduğu, bir tesisin erken uyarıya mı yoksa sessizliğe mi daha çok değer verdiğine bağlıdır.

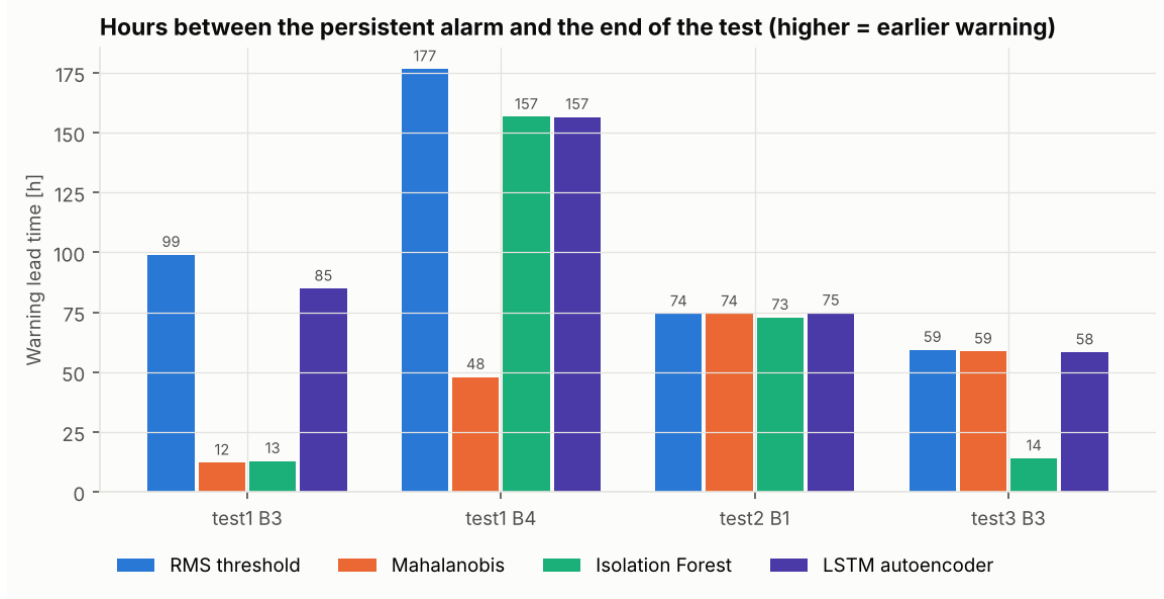
Tablo 5: Her arızalı rulmanda kalıcı alarmın uyarı süresi (testin bitişinden kaç saat önce) ve yanlış alarmlar. Kalın: sütundaki en iyi değer.

Dedektör	test1 (ayrılmış)		test2 (gelişt.)	test3 (ayrılmış)	Yanlış alarm / 1000 rulman-sa
	R3 iç bil.	R4 yuv. el.	R1 dış bil.	R3 dış bil.	
RMS eşiği	99,0	176,6	74,3	59,3	5,34
Mahalanobis	12,2	48,0	74,3	58,7	0,67
Isolation Forest	12,9	156,6	72,8	14,0	0,00
LSTM otokodlayıcı	84,9	156,6	74,7	58,3	12,67

İkinci tablo yanlış alarmları test test ayırır. Satırlardaki sayılar, her testin sağlıklı kabul edilen penceresinde kaç ayrı yanlış alarm epizodu (alarmın açılıp kapandığı ayrı olay) yaşandığını gösterir. En alttaki iki satır bu pencerenin testin hangi saatleri arasında olduğunu ve büyüklüğünü (dört rulmanın toplamı olarak rulman-saat cinsinden) verir. Üç testin toplamı yaklaşık 2 999 rulman-saattir; örneğin RMS eşiğinin toplam 16 yanlış alarmı $16/2\,999 \times 1\,000 \approx 5,34$, LSTM otokodlayıcının toplam 38 yanlış alarmı ise $\approx 12,67$ eder; ana tablodaki son sütun budur.

Tablo 6: Test başına yanlış alarm bölümleri ve değerlendirme penceresinin boyutu.

Dedektör	test1	test2	test3
RMS eşiği	15	0	1
Mahalanobis	0	0	2
Isolation Forest	0	0	0
LSTM otokodlayıcı	4	2	32
Sağlıklı pencere (saat)	434–632	57–82	380–908
Sağlıklı pencere (rulman-saat)	790	97	2 111



Şekil 5: Her arızalı rulman ve dedektör için kalıcı alarmın uyarı süresi.

Bu grafik nasıl okunur?

Bu, Tablo 5'deki uyarı sürelerinin çubuk grafik hâlidir. **Yatay eksen**de bozulan dört rulman gruplanmıştır: “test1 B3” test 1’in 3 numaralı rulmanı demektir (“B” İngilizce *bearing*, yani rulman). **Dikey eksen** uyarı süresidir, saat cinsinden (“Warning lead time [h]”): kalıcı alarm ile testin bitişi arasındaki süre. **Çubuk ne kadar uzunsa uyarı o kadar erkendir**. Her grupta dört renkli çubuk dört yöntemi gösterir: mavi RMS eşiği, turuncu Mahalanobis, yeşil Isolation Forest, mor LSTM otokodlayıcı. Çubukların üstündeki sayılar tam saattir (yuvarlanmış).

Ne görüyoruz? Test 2’de dört çubuk pratikte eşittir (73–75 sa). Ayrılmış testler dedektörleri birbirinden ayırır. Test 1’de RMS temel modeli ve LSTM otokodlayıcı en erken uyarıyı verir (85–177 sa); Mahalanobis uzaklığı ise bitişten yalnızca 12–48 sa önce uyarır. Test 3’te üç dedektör yaklaşık 59 sa, Isolation Forest ise yalnızca 14 sa verir.

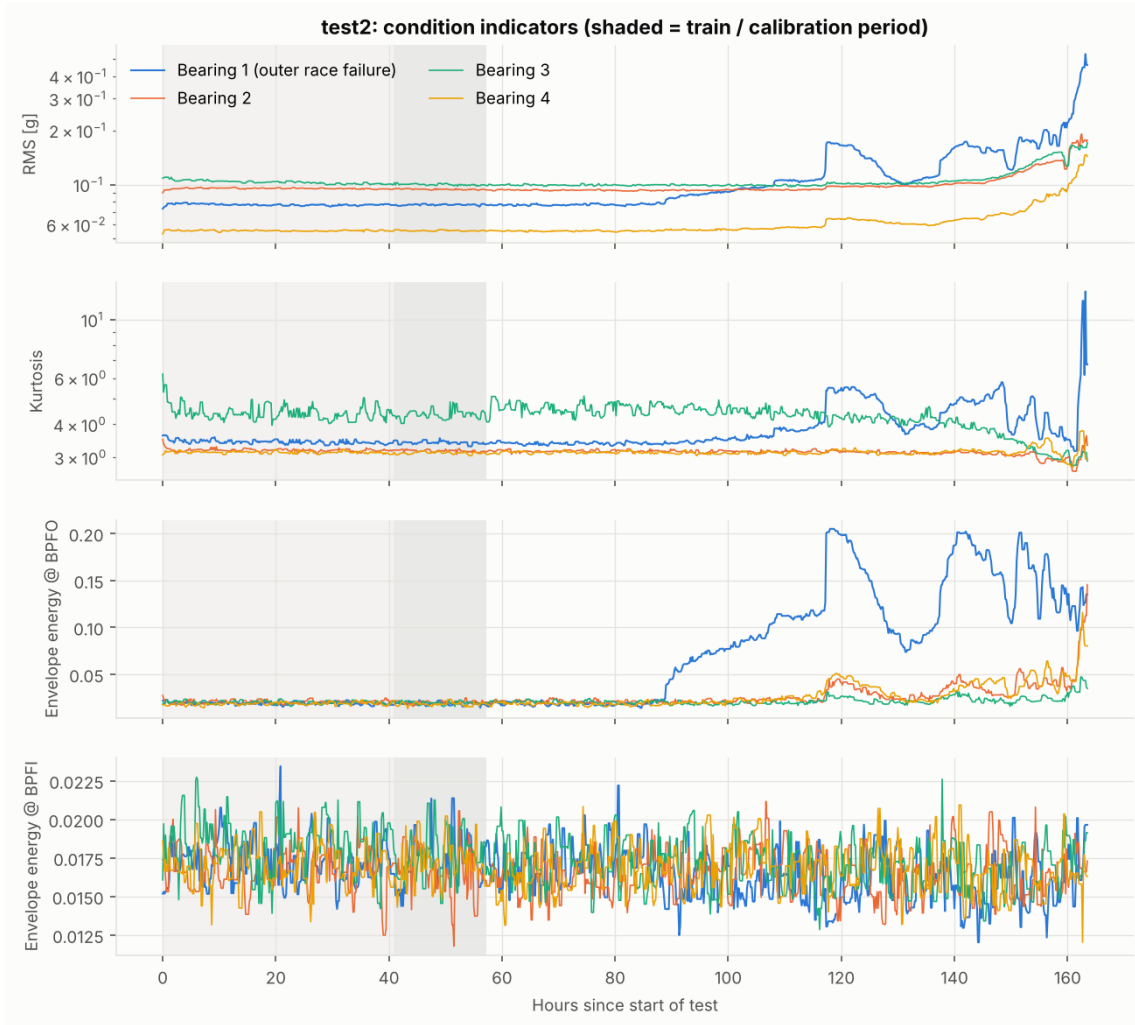
Ne anlama geliyor? Bir hasar temiz biçimde geliştiğinde (test 2) makul her dedektör onu aynı anda görür ve dedektör seçimi önemli değildir. Durum karıştığında (kayıt boşlukları, yeniden başlatmalar) yöntemler arasındaki farklar ortaya çıkar.

Neden önemli? Tablo 6 ile birlikte okunduğunda, en erken uyarı veren dedektörlerin aynı zamanda en çok yanlış alarm üreten dedektörler olduğu görülür; bedava öğle yemeği yoktur (yani bir şeyi kazanmak için başka bir şeyden vazgeçmek gerekir). Duman dedektörü benzetmesindeki hassasiyet uyarı tam olarak budur.

6.2 Test 2 (geliştirme kümesi)

Basitçe

Test 2, sistemi ayarlarken kullandığımız “çalışma deneyi”dir. Yaklaşık 7 gün sürmüştür ve sonunda 1 numaralı rulmanın dış halkası bozulmuştur. Bu deneyde hasar temiz ve düzenli geliştiği için sistemin en iyi koşullarda nasıl çalıştığını gösterir. Aşağıda önce ölçümlerin (öznitelikler) zaman içindeki değişimine, sonra dört dedektörün puanlarına bakıyoruz.



Şekil 6: Test 2'nin durum göstergeleri (5 kayıtlık kayan medyan). Gölge alanlar: eğitim ve kalibrasyon dönemleri.

Bu grafik nasıl okunur?

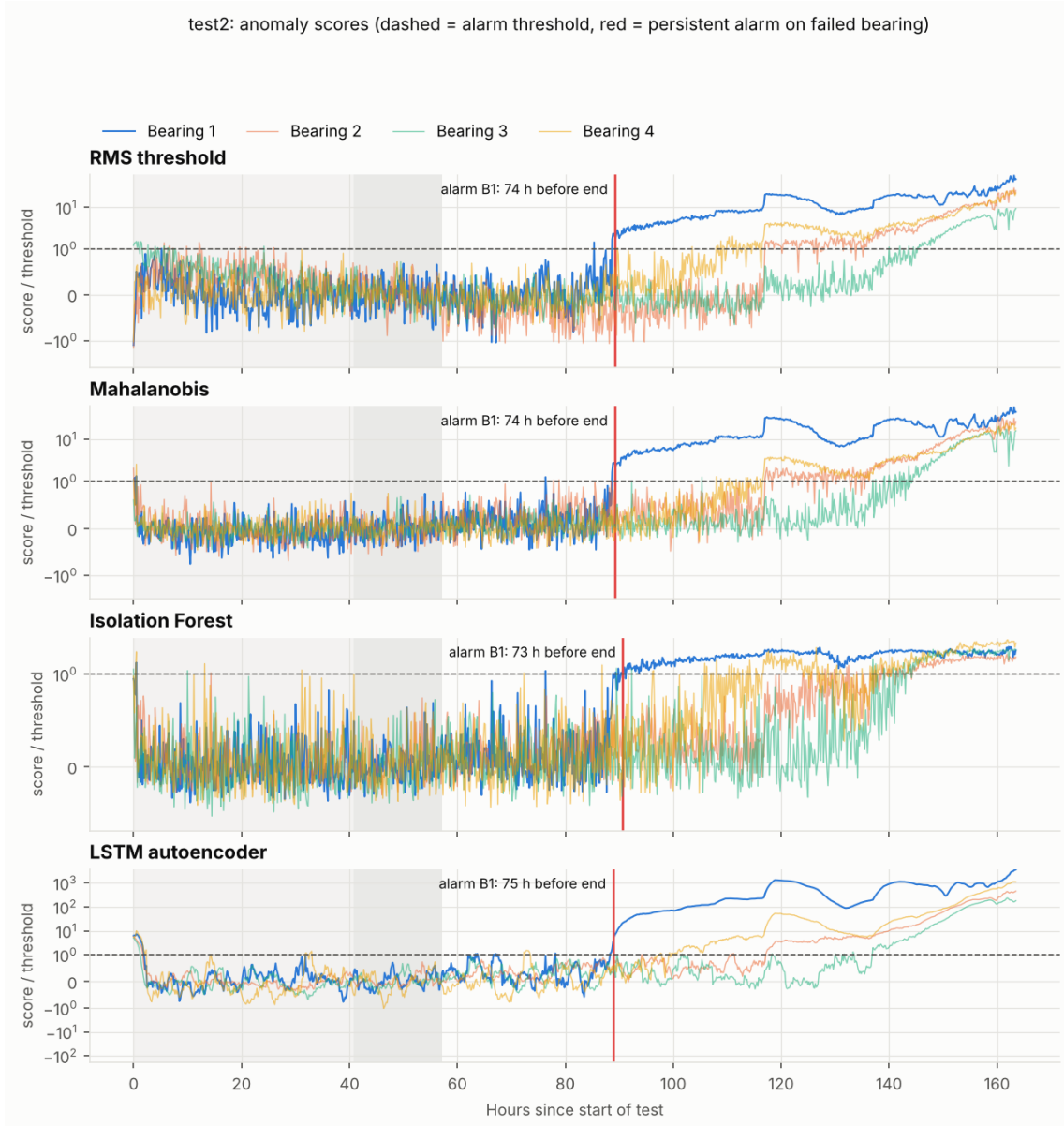
Dört grafik üst üste dizilmiştir ve hepsinin **yatay eksen**i ortaktır: testin başından bu yana geçen süre, saat cinsinden (“Hours since start of test”). Her grafik bir öznelik gösterir; yukarıdan aşağıya: **RMS** (titreşim gücü, g), **basıklık** (Kurtosis; vuruntululuk, birimsiz), **BPFO'daki zarf enerjisi** (dış bilezik hasar ritminin zarf içindeki payı) ve **BPFI'deki zarf enerjisi** (iç bilezik hasar ritminin payı). Her renk bir rulmandır: mavi rulman 1 (test sonunda dış bileziği bozuk bulunan; açıklamada “outer race failure” diye belirtilmiş), turuncu-kırmızı rulman 2, yeşil rulman 3, sarı-turuncu rulman 4. Çizgiler, ani tek tük sıçramaları bastırmak için son 5 kaydın medyanı alınarak yumuşatılmıştır. İlk iki grafiğin dikey eksen **logaritmiktir** ($10^{-1} = 0,1$, $10^0 = 1$, $10^1 = 10$; eşit aralıklar eşit *katlara* karşılık gelir), alttaki iki grafiğinki doğrusaldır. **Açık gri gölge** eğitim dönemini (sistemin normal öğrendiği ilk %25), **koyu gri gölge** kalibrasyon dönemini (alarm sınırının belirlendiği sonraki %10) gösterir.

Ne görüyoruz? İlk 85 saat boyunca dört rulmanın tüm göstergeleri düzdür. Yaklaşık 88. saatte rulman 1'in BPFO zarf enerjisi 0,02'den 0,05'e sıçrar ve 118. saatte yaklaşık 0,2'ye yükselmeye devam eder; RMS ise yalnızca yavaşça artar. Rulman 1'in basıklığı da (darbeler) artar, ancak daha kademeli olarak. BPFI zarf enerjisi tüm rulmanlar için gürültü seviyesinde kalır. Yaklaşık 115. saatten itibaren diğer rulmanların RMS'i ve BPFO enerjisi de artar. Son saatlerde tüm göstergeler dik biçimde yükselir. Rulman 1'in BPFO enerjisinde 130. saat civarında bir “çukur” (geçici düşüş)

vardır.

Ne anlama geliyor? BPFO'daki sıçrama, Şekil 4'de görülen dış bilezik hasarıdır. BPFI'nin düz kalması, hasarın iç bilezikte değil dış bilezikte olmasıyla tutarlıdır. Diğer rulmanlardaki geç artış, onların bozulduğu anlamına gelmez: rulman 1'in titreşimi mil ve gövde üzerinden diğerlerine iletilir (bir masadaki bir bardağa vurduğunuzda yanındakilerin de titreşmesi gibi). 130. saat civarındaki "çukur" oyuk (spall; yüzeyden kopan küçük parçanın bıraktığı çukur) gelişiminde tipiktir: hasarın kenarları aşınıp yumuşadıkça darbeler geçici olarak zayıflar, ardından hasar yayılır.

Neden önemli? Zarf enerjisi, toplam titreşim gücü (RMS) belirgin biçimde artmadan önce ve doğrudan hasarlı parçayı işaret ederek tepki verir. Bu, fiziğe dayalı öznelitliklerin değerini gösterir.



Şekil 7: Test 2 anormali puanlarının her rulmanın eşğine oranı (simetrik log ölçek). Kesikli çizgi: alarm eşği; kırmızı çizgi: arızalı rulmandaki kalıcı alarm.

Bu grafik nasıl okunur?

Dört grafik üst üste dizilmiştir; her biri bir dedektördür (yukarıdan aşağıya RMS eşği, Mahalanobis, Isolation Forest, LSTM otokodlayıcı). **Yatay eksen** testin başından bu yana

geçen saattir. **Dikey eksen** (“score / threshold”) her rulmanın puanını kendi eşikğine göre ölçeklenmiş olarak gösterir: **0** o rulmanın kalibrasyon dönemindeki tipik (medyan) puanıdır, **1** ise alarm eşikğidir. Böylece dört rulman, eşikleri farklı olsa da aynı grafikte karşılaştırılabilir. **Kesikli yatay çizgi** (1 seviyesi) alarm eşikğidir: çizginin üstü “eşik aşıldı” demektir. Dikey eksen **simetrik logaritmik** (symlog) ölçektedir: -1 ile $+1$ arası sıradan (doğrusal) ölçekle, bunun dışı logaritmik ölçekle çizilir; yani $10^0 = 1$, $10^1 = 10$, $10^2 = 100$, $10^3 = 1000$ çizgileri eşit aralıklıdır ve eşikğin 10 ya da 1000 katı puanlar aynı grafiğe sığar. Eksisi tarafta ($-10^0 = -1$ gibi) aynı şey aynalanmıştır. Renkler rulmanlardır: koyu mavi rulman 1 (bozulan), soluk turuncu rulman 2, soluk yeşil rulman 3, soluk sarı rulman 4. **Dikey kırmızı çizgi**, bozulan rulmandaki kalıcı alarmın başladığı andır; yanındaki not (ör. “alarm B1: 74 h before end”) bunun testin bitişinden kaç saat önce olduğunu söyler. Açık ve koyu gri gölgeler yine eğitim ve kalibrasyon dönemleridir.

Ne görüyoruz? Dört dedektör de eğitim, kalibrasyon ve sonraki 30 saat boyunca tüm rulmanları eşikğin altında tutar; ardından rulman 1 89. saatte eşikği aşar ve bir daha altına inmez: bitişten 74 saat önce kalıcı alarm. Puan sıçraması tüm öznelilikleri birleştiren LSTM otokodlayıcıda (iki merteye, yani yaklaşık 100 kat) ve Mahalanobis uzaklığında (yaklaşık $20\times$) en büyük, puanı yapısı gereği doygunluğa ulaşan (belli bir değerin üstüne çıkamayan) Isolation Forest’ta en küçüktür. İki kısa LSTM bölümü (yanlış alarm olarak sayılır) dışında diğer rulmanlar eşikği ancak 109–145. saatlerden sonra ve her zaman rulman 1’den sonra aşar.

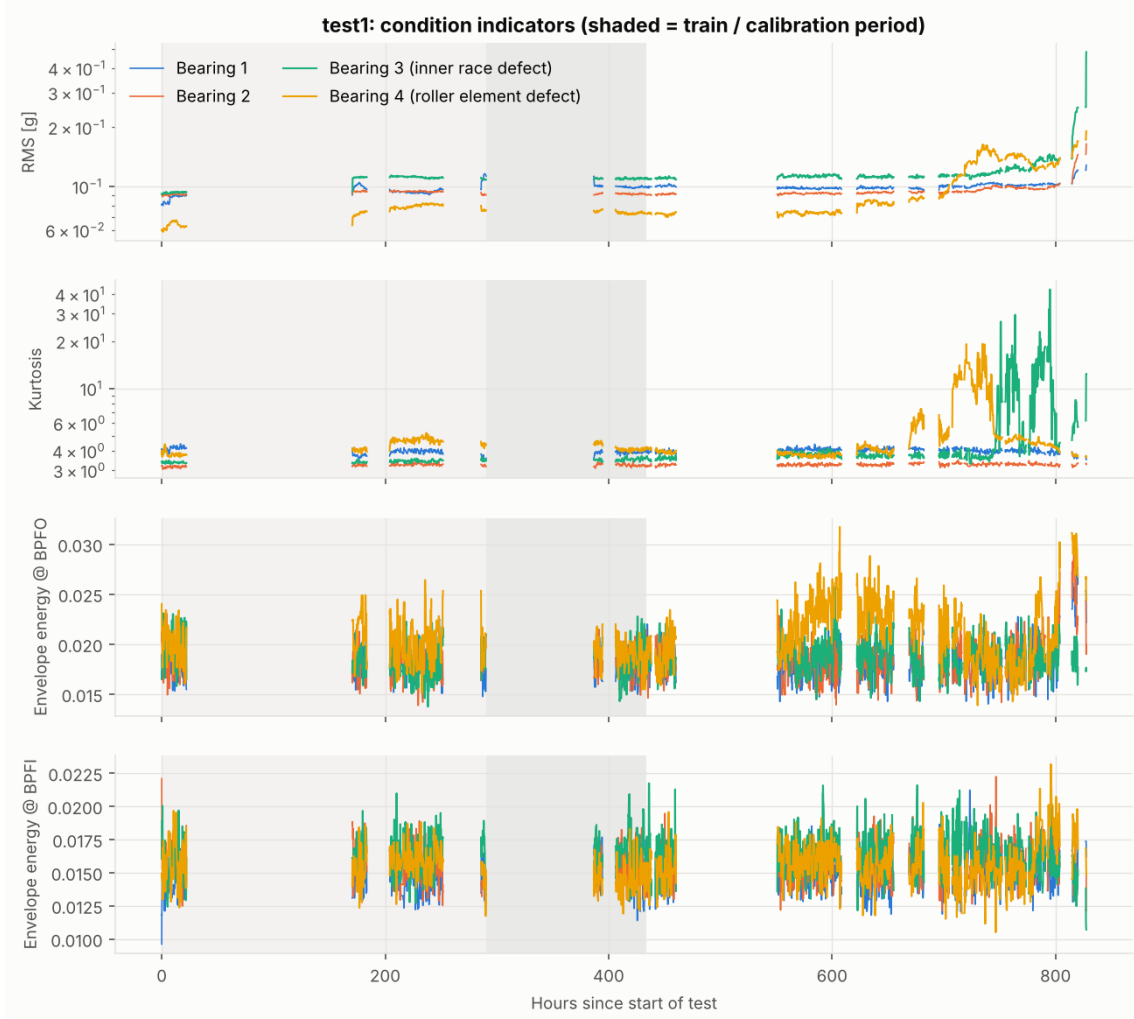
Ne anlama geliyor? Dolayısıyla *ilk alarm veren rulman* kaynağı doğru biçimde gösterir. Tüm dedektörler alarm anında **dış bilezik** hasarı teşhis eder (z-puanları 6,1–7,3; yani dış bilezik ritmi normalinin 6–7 standart sapma üstünde); bu, belgelenmiş arıza tipidir. Rulman 2 ve 4 de sonraki alarmlarında dış bilezik teşhisi alır, çünkü rulman 1’in titreşimini taşırlar.

Neden önemli? Panelin ilk alarm veren rulmanı muhtemel kaynak olarak işaretlemesinin nedeni budur: birden fazla rulman alarm verdiğinde, operatöre “asıl sorun büyük ihtimalle ilk ötende” bilgisi verilir. Ayrıca bu deney, sistemin en iyi koşullarda bakım için yaklaşık üç günlük bir pencere sağladığını gösterir.

6.3 Test 1 (ayrılmış)

Basitçe

Test 1, ayarlar yapılırken hiç bakılmamış ilk “sınav” deneyidir. Yaklaşık 35 gün sürmüştür, ama kayıt günlerce süren boşluklarla parça parça alınmıştır. Sonunda iki rulman bozulmuştur: rulman 3’ün iç halkası ve rulman 4’ün makaraları. Bu deney sistem için zordur; sonuçlar hem güçlü hem zayıf yanları gösterir.



Şekil 8: Test 1'in durum göstergeleri. Çizgi kesintileri kayıt boşluklarıdır.

Bu grafik nasıl okunur?

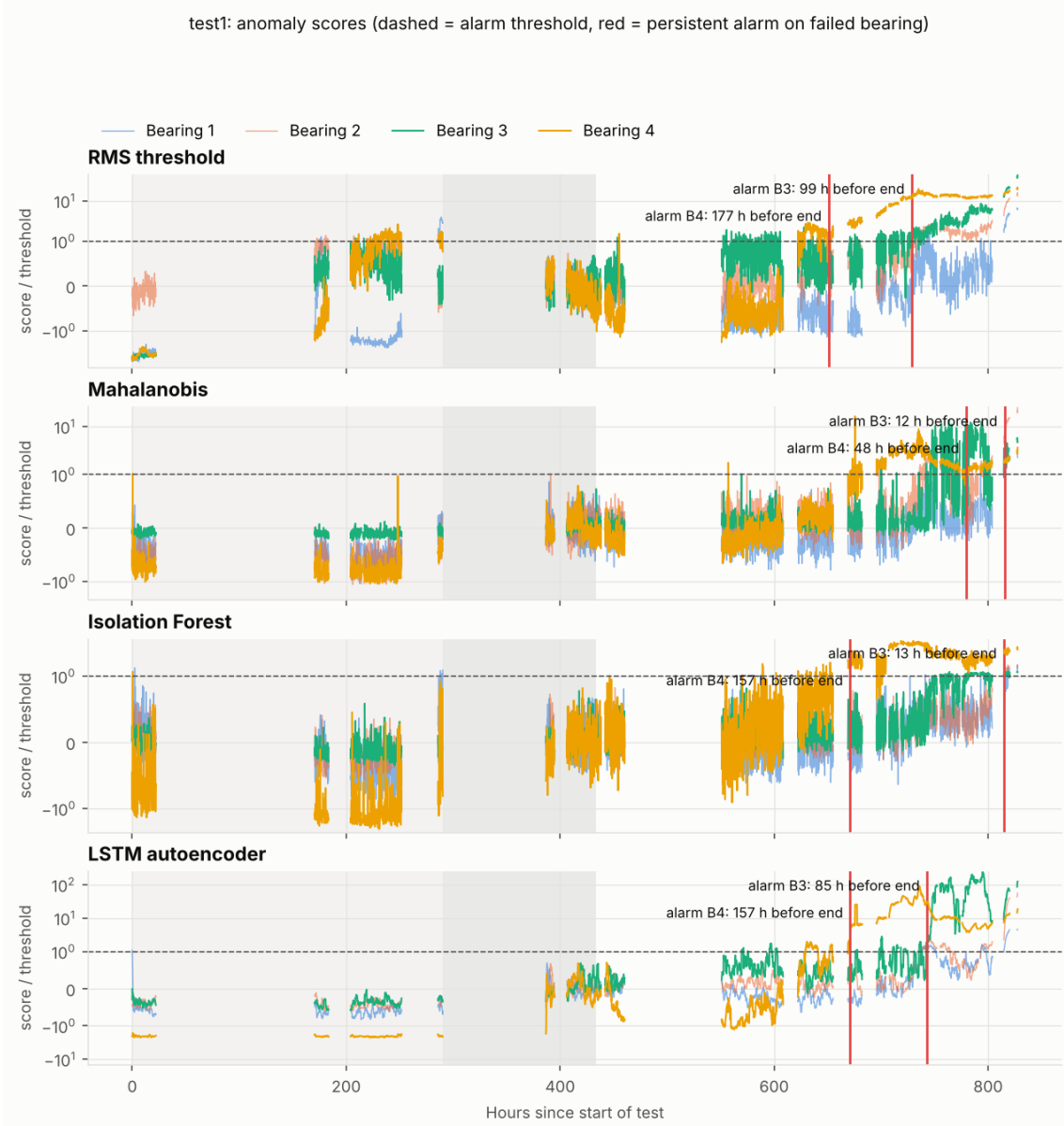
Düzen, Şekil 6 ile aynıdır: yatay eksen testin başından bu yana geçen saat; yukarıdan aşağıya RMS (g, logaritmik eksen), basıklık (logaritmik eksen), BPFO'daki ve BPFI'deki zarf enerjisi (doğrusal eksen); açık gri gölge eğitim, koyu gri gölge kalibrasyon dönemi; çizgiler 5 kayıtlık kayan medyanla yumuşatılmış. Renkler: mavi rulman 1, turuncu-kırmızı rulman 2, **yeşil rulman 3** (test sonunda iç bileziği bozuk; “inner race defect”), **sarı-turuncu rulman 4** (test sonunda makarası bozuk; “roller element defect”). Bu testte her rulmanda iki sensör vardır; grafik bunlardan birincisini gösterir. **Çizgilerin kesildiği yerler** kayıt alınmayan dönemlerdir (ör. yaklaşık 25. ile 170. saatler arası); veri yoktur, makine çizgi yokken ne yaptığı bilinmez.

Ne görüyoruz? Test 1 en zor kayıttır: altı güne varan boşluklarla parçalanmıştır ve eğitim dönemi (ilk 539 kayıt, 290. saate kadar) kayıt parçaları arasında birkaç basamak değişimi (bir parçadan diğerine seviyenin aniden kayması) içerir. İki arızalı rulmanın bozulması basıklıkta açıkça görülür: rulman 4 yaklaşık 680. saatten itibaren (basıklık 20'ye kadar), rulman 3 ise yaklaşık 720. saatten itibaren (40'a kadar) darbeleri hâle gelir; bu, RMS'leri artmadan çok öncedir. BPFI'deki zarf enerjisi (rulman 3'ün iç bilezik hasarı) hiç artmaz, rulman 4'ün BPFO enerjisi de yalnızca dalgalanır.

Ne anlama geliyor? “Sağlıklı” tek ve kararlı bir durum değildir; sistem normal öğrenirken zaten birkaç farklı “normal” görmüştür. Basıklığın erken yükselmesi, lokal hasarın önce seyrek darbeler olarak ortaya çıktığını doğrular. Zarf özniteliklerinin bu kez işe yaramamasının nedeni şudur:

iç bilezik ve yuvarlanma elemanı hasarları yük bölgesine (rulmanın yükü taşıyan alt kısmına) girip çıktıkları için zarf enerjilerini tek bir çizgide toplamak yerine $\pm f_r$ ve kafes frekansındaki yan bantlara (ana ritmin hemen sağına ve soluna düşen komşu ritimlere) yayarlar; harmoniklerin çevresindeki ± 3 Hz'lik pencere bu enerjinin çoğunu kaçırrır.

Neden önemli? Teşhis yöntemimiz dış bilezik hasarlarında güçlüdür, ancak iç bilezik ve makara hasarları için geliştirilmesi gerekir (bkz. Bölüm 9).



Şekil 9: Test 1 anomali puanlarının her rulmanın eşiğine oranı.

Bu grafik nasıl okunur?

Düzen, Şekil 7 ile aynıdır: her satır bir dedektör; yatay eksen saat; dikey eksen puanın eşiğe göre ölçeklenmiş hâli (0 = tipik sağlıklı puan, 1 = eşik), simetrik logaritmik ölçekte; kesikli yatay çizgi alarm eşiği; açık ve koyu gri gölgeler eğitim ve kalibrasyon. Bu testte iki rulman bozulduğu için her grafikte **iki dikey kırmızı çizgi** vardır: biri rulman 3'ün (B3), biri rulman 4'ün (B4) kalıcı alarminin başlangıcı; notlar hangisinin testin bitişinden kaç saat önce olduğunu söyler. Koyu renkli çizgiler bozulan rulmanlar (yeşil rulman 3, sarı-turuncu rulman 4), soluk çizgiler

sağlam kalan rulmanlardır (mavi rulman 1, turuncu-kırmızı rulman 2). Çizgi kesintileri kayıt boşluklarıdır.

Ne görüyoruz? Rulman 4, üç dedektör tarafından bitişten 157–177 saat önce (deneyin %19–21’i) tespit edilir ve her dedektör için ilk alarm veren rulmanlar iki arızalı rulmandır. RMS temel modeli en erken uyarıyı verir, ancak sağlıklı kabul edilen pencere içinde (434–632. saatler) eşiği etrafında salınarak rulman 3’te (554. saatten itibaren) ve rulman 4’te (623. saatten itibaren) toplam 15 alarm bölümü üretir. Mahalanobis uzaklığı ve Isolation Forest hiç yanlış alarm vermez, ancak rulman 3’teki kalıcı alarmları bitişten yalnızca 12–13 sa önce başlar.

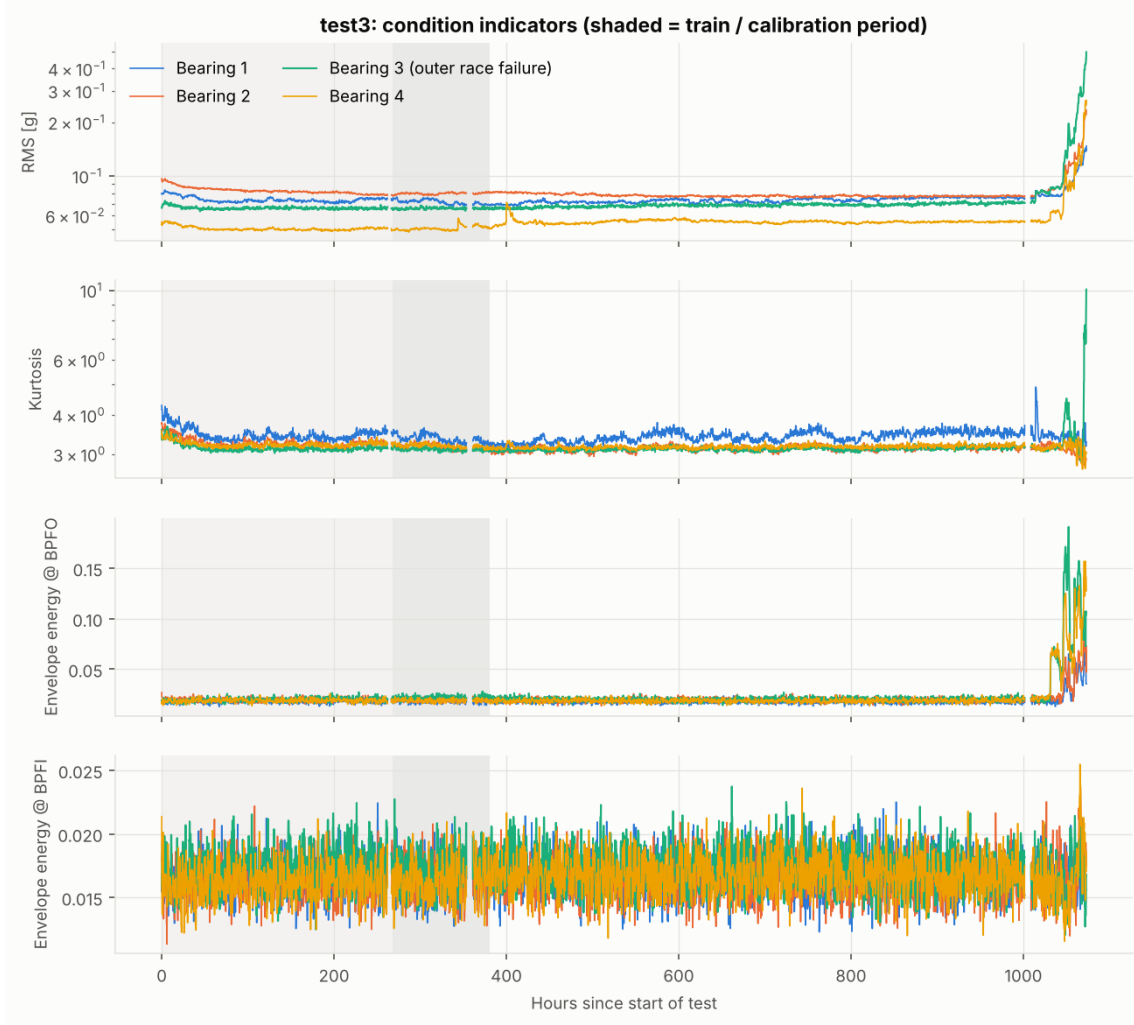
Ne anlama geliyor? Kaynak tespiti doğrudur: önce bozulan rulmanlar alarm verir. RMS eşiğinin “yanlış” alarmları tam da daha sonra arızalanan iki rulmanda olduğu için, bu bölümlerin bir kısmı aslında yanlış alarm değil, bozulmanın ilk işaretleri olabilir; etiketli bir başlangıç olmadığından değerlendirme bunları temkinli biçimde yanlış alarm sayar. Mahalanobis ve Isolation Forest de aslında eşiği çok daha önce aşmışlardır (rulman 3 için ilk alarm 747. ve 756. saatlerde), fakat puan yeniden eşiğin altına indiği için temkinli “kalıcı alarm” tanımı bu uyarıları dışarıda bırakır. Bu testte en iyi denge LSTM otokodlayıcıdadır (85 sa ve 157 sa, 4 yanlış alarm bölümü).

Neden önemli? Sayılar tek başına okunduğunda yöntemlere haksızlık edilebilir: “geç uyarı” görünen bir yöntem aslında erken ama aralıklı uyarımış olabilir. Ayrıca, yukarıda açıklanan nedenle hiçbir dedektör test 1’de güvenilir bir arıza tipi teşhisi üretmez.

6.4 Test 3 (ayrılmış)

Basitçe

Test 3, ikinci “sınav” deneyidir ve en uzundur (yaklaşık 44 gün). Sonunda rulman 3’ün dış halkası bozulmuştur. Bu deneyin özelliği, makinenin birkaç kez durdurulup yeniden çalıştırılmasıdır. Her yeniden çalıştırmadan sonra makinenin “normal” sesi biraz değişir. Bu, sistemin en önemli zayıf noktasını ortaya çıkarır: başta öğrendiği normale takılı kalır ve yeni normal bir sorun sanabilir.



Şekil 10: Test 3'ün durum göstergeleri.

Bu grafik nasıl okunur?

Düzen, Şekil 6 ile aynıdır: yatay eksen saat (0–yaklaşık 1 070); yukarıdan aşağıya RMS (g, logaritmik), basıklık (logaritmik), BPFO'daki ve BPFI'deki zarf enerjisi (doğrusal); açık gri gölge eğitim, koyu gri gölge kalibrasyon; 5 kayıtlık kayan medyanla yumuşatılmış çizgiler. Renkler: mavi rulman 1, turuncu-kırmızı rulman 2, **yeşil rulman 3** (test sonunda dış bileziği bozuk; “outer race failure”), sarı-turuncu rulman 4. Çizgilerdeki kısa kesintiler (ör. yaklaşık 354. ve 1 002. saatler) makinenin durdurulduğu dönemlerdir.

Ne görüyoruz? Test 3 1 000 saat boyunca neredeyse hiç bozulma belirtisi göstermez. İki olay önemlidir. Birincisi, 354. saat civarındaki 7,5 saatlik duruştan sonra tüm rulmanların RMS'i hafifçe kayar ve rulman 4 yaklaşık 400. saatte kısa bir sıçrama gösterir. İkincisi, 1 002. saatteki 7,3 saatlik duruştan sonra tüm göstergeler değişir ve yaklaşık 1 040. saatten itibaren rulman 3'ün (ve iletim yoluyla rulman 4'ün) BPFO zarf enerjisi keskin biçimde yükselir; son 20 saatte bunu RMS ve basıklık izler.

Ne anlama geliyor? İlk olayda makine biraz farklı bir koşulda yeniden çalışmaya başlamıştır; bu bir hasar değil, “yeni bir normal”dir. İkinci olaydaki BPFO yükselişi gerçek hasardır. Dolayısıyla rulman 3'teki dış bilezik hasarı 44 günlük bir deneyin yalnızca son %5'inde görünür hâle gelir.

Neden önemli? Bazı hasarlar çok geç ve hızlı gelişir; hiçbir yöntem var olmayan bir işareti erkenden yakalayamaz. Ayrıca duruşlar, öğrenilmiş “normal”i geçersiz kılabilir.



Şekil 11: Test 3 anomali puanlarının her rulmanın eşğine oranı.

Bu grafik nasıl okunur?

Düzen, Şekil 7 ile aynıdır: her satır bir dedektör; yatay eksen saat; dikey eksen puanın eşığe göre ölçeklenmiş hâli (0 = tipik sağlıklı puan, 1 = eşik), simetrik logaritmik ölçekte; kesikli yatay çizgi eşik; gri gölgeler eğitim ve kalibrasyon. Koyu yeşil çizgi bozulan rulman 3, soluk çizgiler diğer rulmanlardır (mavi rulman 1, turuncu-kırmızı rulman 2, sarı rulman 4). Dikey kırmızı çizgi rulman 3'ün kalıcı alarmının başladığı an; not, testin bitişinden kaç saat önce olduğunu söyler.

Ne görüyoruz? RMS, Mahalanobis ve LSTM dedektörleri rulman 3'te bitişten yaklaşık 59sa önce kalıcı alarm verir; ancak bu alarm duruştan sonra kaydın yeniden başlamasından (1009. saat) yaklaşık 5 saat sonra başlar ve rulman 1'deki alarmlarla çakışır. Isolation Forest ise BPFO enerjisi sıçradığında, bitişten 14sa önce alarm verir ve 11,4 z-puanıyla **dış bilezik** hasarı teşhis eder. 354. saatten sonra LSTM otokodlayıcının puanları sık sık eşğin etrafında dolaşır.

Ne anlama geliyor? İlk üç dedektörün alarmının bir kısmı hasara değil yeniden başlatmaya verilen tepkidir ve bu dedektörler hangi rulmanın arızalandığını ayırt edemez (rulman 1 ve 3 aynı

kayıtta alarm verir). Alarmı açıkça hasardan kaynaklanan tek dedektör Isolation Forest'tır. 354. saatten sonraki rejim değişimi (makinenin çalışma koşulunun kalıcı olarak biraz değişmesi) yanlış alarmların ana kaynağıdır: tüm özniteliklerin ortak değişimini modelleyen LSTM otokodlayıcı yeni temel seviyeyi anormal olarak yorumlar ve çoğu rulman 1'de olmak üzere 32 bölüm üretir.

Neden önemli? Bu, deneyin başında bir kez eğitilen bir modelin en önemli sınırlamasıdır. Gerçek bir tesiste makineler bakım için durdurulur, parçalar değişir; sistemin “normal”i bu olaylardan sonra güncellemesi gerekir.

6.5 Bulguların özeti

Basitçe

Altı cümlelik özet: Hasar düzenli geliştiğinde sistem yaklaşık üç gün önceden uyarır ve hasarın yerini doğru söyler. Erken uarmak ile boşuna alarm vermemek arasında bir seçim yapmak gerekir. En basit yöntem (titreşim gücüne bakmak) şaşırtıcı derecede iyidir. İlk alarm veren rulman genellikle asıl sorunlu rulmandır. Hasar yeri tahmini dış halka hasarlarında iyi, diğerlerinde henüz zayıftır. Makine durdurulup yeniden çalıştırıldığında sistem şaşırabilir.

1. **Hasar temiz biçimde geliştiğinde erken uyarı çalışır.** Test 2'de tüm dedektörler bitişten 74sa (deneyin %45'i) önce uyarır ve doğru arıza tipini belirtir.
2. **Uyarı süresi ile yanlış alarm arasında ödünleşim vardır.** RMS temel modeli ve LSTM otokodlayıcı en erken uyarır; Isolation Forest hiç yanlış alarm vermemiş ama dört arızanın ikisinde geç uarmıştır; Mahalanobis uzaklığı çok az yanlış alarmla arada yer alır.
3. **Güçlü bir temel model önemlidir.** Basit RMS eşiği uyarı süresinde rekabetçidir. Yalnızca üç deney içeren bir veri setinde derin model üstünlük kurmaz.
4. **İlk alarmla kaynak tespiti** test 1 ve test 2'de güvenilirdir, ancak test 3'teki yeniden başlatmadan sonra değildir.
5. **Zarf teşhisi dış bilezik hasarları için güvenilirdir** (test 2 ve test 3), ancak mevcut dar bantlı öznitelikle test 1'in iç bilezik ve yuvarlanma elemanı hasarlarını tanımaz.
6. **Çalışma koşulu değişiklikleri** (yeniden başlatma, bakım) geçerliliğe yönelik ana tehdittir: ömrün başında bir kez eğitilen bir model yeni bir normal anomali olarak görür.

7 Etkileşimli Panel (Dashboard)

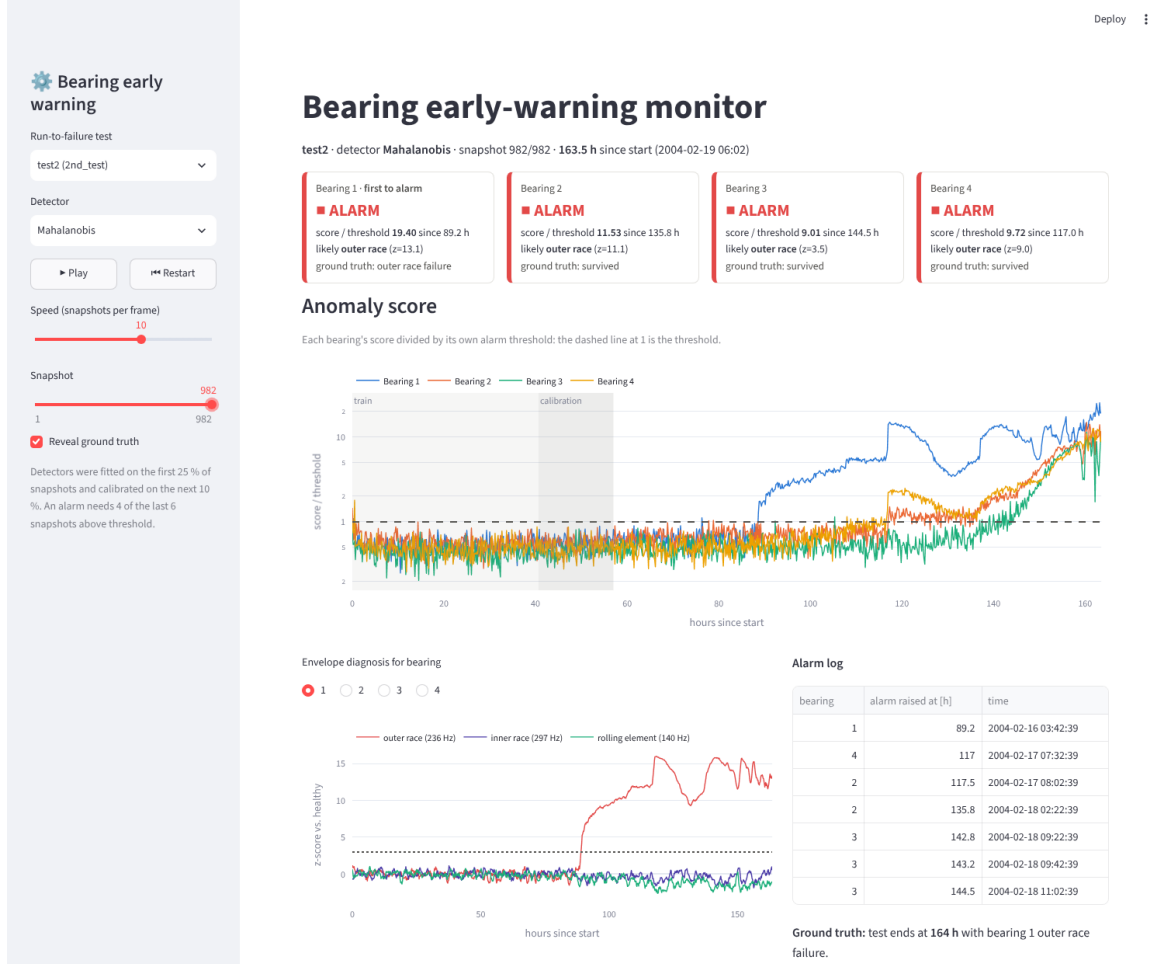
Basitçe

Panel, sistemin bir fabrikadaki operatörün ekranında nasıl görüneceğini gösteren bir web sayfasıdır. Geçmişteki bir deneyi, sanki makine şu anda çalışıyormuş gibi baştan sona “oynatır”; bir maçın tekrarını izlemek gibi. Operatör her rulmanın durumunu renkli kartlarda görür, puanların zaman içindeki seyrini izler ve alarm çaldığında teşhisin hangi kanıta dayandığını görebilir.

Sistemin bir operatöre nasıl görüneceğini göstermek için `app/dashboard.py` herhangi bir testi makineden canlı veri geliyormuş gibi yeniden oynatır (Şekil 12). Kullanıcı bir test ve bir dedektör seçip *Play*'e (Oynat) basar; panel kayıt kayıt ilerler ve şunları gösterir:

- her rulman için renk, simge ve etiketle bir durum kartı (sağlıklı / eşik üstünde / alarm), puan/eşik oranı, mevcut alarmın başlangıcından bu yana geçen süre, muhtemel kaynak için *ilk alarm veren* (first to alarm) işareti ve teşhis edilen arıza tipi;

- eğitim ve kalibrasyon dönemleri gölgelenmiş, o ana kadarki tüm rulmanların anomali puanı grafiği;
- seçilen rulman için BPFO, BPFI ve BSF'deki zarf z-puanları;
- bir alarm günlüğü ve istenirse gerçek durum (hangi rulmanın nasıl arızalandığı).



Şekil 12: Test 2'nin sonunda, Mahalanobis dedektörüyle ve gerçek durum gösterilirken panel.

Bu grafik nasıl okunur?

Bu bir ekran görüntüsüdür. **Sol kenar çubuğu** ayarlardır: test seçimi ("Run-to-failure test"), dedektör seçimi ("Detector"), Oynat/Baştan başlat düğmeleri ("Play", "Restart"), oynatma hızı (her adımda kaç kayıt ilerleneceği), hangi kayıta olunduğunu gösteren kaydırıcı ("Snapshot", burada 982/982, yani testin sonu) ve gerçek durumu gösterme kutucuğu ("Reveal ground truth"). **Üstteki dört kart** dört rulmanın anlık durumudur: kırmızı "ALARM" yazısı alarmı, "score / threshold" puanın eşik kaç katı olduğunu ve alarmın hangi saatten beri sürdüğünü, "likely outer race ($z=...$)" teşhisi (muhtemel dış bilezik hasarı ve ne kadar belirgin olduğu), "ground truth" ise testin sonunda gerçekte ne olduğunu ("outer race failure" = dış bilezik arızası, "survived" = sağlam kaldı) gösterir. Rulman 1'in kartında "first to alarm" (ilk alarm veren) işareti vardır. **Ortadaki grafik** ("Anomaly score") Şekil 7'in canlı hâlidir: yatay eksen saat, dikey eksen puan/eşik (logaritmik ölçek), kesikli çizgi eşik (1), gri alanlar eğitim ("train") ve kalibrasyon ("calibration") dönemleri, renkler rulmanlar (mavi 1, turuncu 2, yeşil 3, sarı 4). **Sol alttaki grafik** ("Envelope diagnosis for bearing") seçilen rulman (burada 1) için üç hasar ritminin z-puanını zamana göre gösterir: kırmızı dış bilezik (236 Hz), mor iç

bilezik (297 Hz), yeşil yuvarlanma elemanı (140 Hz); noktalı yatay çizgi teşhis için kullanılan 3 sınırdır. **Sağ alttaki tablo** (“Alarm log”) her alarmın hangi rulmanda, kaçınıcı saatte ve hangi tarih-saatte başladığını listeler; en altta gerçek durum özeti yazar.

Ne görüyoruz? Kartlar test 2’nin hikâyesini tek satırda özetler: rulman 1 belirgin bir dış bilezik imzasıyla ($z = 13,1$) ilk alarm veren rulmandır (89,2. saatten beri); diğerleri 28–55 saat sonra onu izler. Zarf grafiği, rulman 1’in BPFO z-puanının 90. saat civarında 0’dan 10’un üstüne sıçradığını, BPFI ve BSF’nin ise düz kaldığını gösterir.

Ne anlama geliyor? Panel yalnızca “alarm” demekle kalmaz; hangi rulmanın muhtemel kaynak olduğunu ve neden dış bilezik dediğini de gösterir: teşhisin arkasındaki kanıt operatör tarafından görülebilir.

Neden önemli? Bir operatör, nedenini göremediği bir alarmı güvenmekte zorlanır. Kanıtı görünür kılmak, sistemin pratikte kullanılabilmesi için gereklidir.

Aşağıdaki kod parçası panelin iki temel işini yapar: hangi rulmanın muhtemel kaynak olduğuna karar vermek ve her rulmanın kartındaki durumu belirlemek; ayrıca oynatmayı ilerletir.

Kod 9: Muhtemel kaynak ve her rulmanın durumu (app/dashboard.py’den alıntı).

```

1 # The first bearing to raise an alarm is the likely source: once it degrades, its
2 # vibration travels through the shaft and housing and lifts its neighbours too.
3 onsets = {b: g.loc[g["alarm"], "hours"].min() for b, g in seen.groupby("bearing") if
4           g["alarm"].any()}
5 source = min(onsets, key=onsets.get) if onsets else None
6
7 for col, (b, g) in zip(cols, seen.groupby("bearing")):
8     last = g.iloc[-1]
9     state = "alarm" if last.alarm else ("watch" if last.score > last.threshold else "ok")
10    defect, zval = diagnose(zs[b].loc[:now]) # mean z of the last 6 snapshots
11    ...
12
13 # Playback: advance the slider and rerun the script
14 if st.session_state.playing and idx < n:
15     st.session_state["advance_to"] = min(n, idx + speed)
16     time.sleep(0.15)
17     st.rerun()

```

Kod ne yapıyor? Satır 1–2’deki yorum fikri açıklar: ilk bozulan rulmanın titreşimi komşularına da yayılır, bu yüzden ilk alarm veren muhtemel kaynaktır. Satır 3, şimdiye kadar görülen veride (seen) alarm vermiş her rulman için ilk alarm saatini bulur. Satır 4 bunların en erkenini “kaynak” seçer. Satır 6–9 her rulman için bir kart hazırlar: son kayıttaki alarm açıksa “alarm”, alarm yok ama puan eşik üstündeyse “watch” (izlemede), değilse “ok” (sağlıklı) der (satır 8) ve son 6 kaydın zarf z-puanlarından teşhisi hesaplar (satır 9). Satır 12–15 oynatmayı sağlar: “Play” açıksa kaydırıcıyı seçilen hız kadar ilerletir, kısa bir süre (0,15 s) bekler ve sayfayı yeniden çizer; bu döngü bir film gibi akış hissi verir.

Panel yalnızca depoya eklenmiş sonuç dosyalarını okur, bu nedenle ham veri olmadan çalışır: **make dashboard** komutu paneli <http://localhost:8501> adresinde (kendi bilgisayarınızda açılan bir web sayfası olarak) açar.

8 Yazılım Mühendisliği ve Yeniden Üretilebilirlik

Basitçe

Bilimsel bir sonuç, başkaları tarafından tekrarlanabildiği ölçüde değerlidir. Bu bölüm, projenin herkesin kendi bilgisayarında aynı sonuçları birkaç komutla tekrar elde edebileceği şekilde nasıl düzenlendiğini anlatır: dosyaların nerede durduğu, ayarların tek bir dosyada toplanması, çalıştırma komutları, kodun doğru çalıştığını otomatik kontrol eden testler ve değişikliklerin kayıt altında tutulması.

8.1 Depo yapısı

Basitçe

Projenin klasör düzeni, iyi düzenlenmiş bir dolap gibidir: ayarlar bir çekmecede, asıl kod bir çekmecede, tek seferlik çalıştırılan betikler bir çekmecede, sonuçlar ve grafikler başka bir çekmecede. Aşağıdaki liste her klasörün ve dosyanın ne işe yaradığını tek satırda söyler.

1	configs/default.yaml	splits, thresholds, alarm rule, model hyper-parameters
2	src/bearing_efw/	
3	dataset.py	test metadata, bearing geometry, defect frequencies, loader
4	features.py	time, spectral and envelope features per snapshot
5	preprocess.py	per-bearing matrices, log + standardisation, splits
6	models.py	RMS, Mahalanobis, Isolation Forest, LSTM autoencoder
7	alarm.py	thresholding, k-of-n debouncing, alarm episodes
8	plots.py	report figures
9	scripts/	download_data, build_features, run_experiment, make_report_figures
10	app/dashboard.py	Streamlit replay dashboard
11	data/features/	extracted features (committed, 3 MB)
12	reports/	results.md, summary.csv, scores.csv.gz, figures/
13	notebooks/	01_walkthrough.ipynb
14	tests/	unit tests and dashboard smoke tests
15	report/	this report (LaTeX sources and PDFs, English and Turkish)

Liste ne diyor? configs/default.yaml tüm ayarları tutar. src/bearing_efw/ asıl kütüphane-dir: dataset.py test bilgileri, rulman ölçüleri ve dosya okuma; features.py 18 öz niteliğin hesabı; preprocess.py logaritma, ölçekleme ve dönemlere bölme; models.py dört dedektör; alarm.py eşik ve alarm kuralları; plots.py rapor grafikleri. scripts/ veriyi indiren, öz nitelikleri çıkaran, deneyi çalıştıran ve grafikleri üreten betiklerdir. app/dashboard.py paneldir. data/features/ çıkarılmış öz nitelikleri (3 MB), reports/ sonuç tablolarını ve grafikleri, notebooks/ adım adım açıklamalı bir defteri, tests/ otomatik testleri, report/ ise bu raporun kaynak dosyalarını içerir.

8.2 Yapılandırma

Basitçe

Bütün ayarlar (dönemlerin uzunluğu, eşik kuralı, alarm kuralı, yapay sinir ağının boyutu) kodun içine dağılmak yerine tek bir metin dosyasında durur. Böylece bir ayarı değiştirmek için kodu açmak gerekmez; bir tarifteki malzeme miktarlarını değiştirmek gibi, sadece o dosyadaki sayıyı değiştirmek yeter.

Deneydeki tüm seçimler tek bir YAML dosyasında (okunması kolay bir ayar dosyası biçimi) tutulur; böylece deney kod değiştirmeden değiştirilebilir:

```
1 split:      {train_end: 0.25, calib_end: 0.35}
2 healthy_end: {test1: 0.60, test2: 0.50, test3: 0.85}
```

```

3 threshold: {quantile: 0.995, margin: 0.5}
4 alarm:     {k: 4, n: 6}
5 models:
6   isolation_forest: {n_estimators: 300, seed: 0}
7   lstm_ae: {window: 12, hidden: 32, latent: 8, epochs: 60, lr: 0.002,
8             batch_size: 64, noise: 0.1, seed: 0}

```

Dosya ne diyor? Satır 1: eğitim kayıtların ilk %25’inde biter, kalibrasyon %35’te biter. Satır 2: yanlış alarmların sayıldığı “sağlıklı” pencere test 1, 2 ve 3’te sırasıyla kayıtların %60, %50 ve %85’inde biter. Satır 3: eşik 0,995 kuantili ve 0,5 payla hesaplanır. Satır 4: alarm için son 6 kaydın 4’ü eşiği aşmalıdır. Satır 5–8: Isolation Forest 300 ağaç kullanır; LSTM otokodlayıcı 12 kayıtlık pencere, 32 gizli birim, 8 sayılık özet, 60 tur eğitim, 0,002 öğrenme oranı, 64’lük gruplar ve 0,1 gürültü kullanır. `seed: 0`, rastgeleliğin her çalıştırmada aynı olmasını sağlar.

8.3 Çalıştırma

Basitçe

Projeyi çalıştırmak için birkaç kısa komut yeterlidir. Her komut bir düğme gibidir: biri gerekli kütüphaneleri kurar, biri deneyi yaklaşık iki dakikada baştan sona çalıştırır, biri paneli açar, biri testleri çalıştırır. Ham veriyi indirmek ve öznitelikleri yeniden hesaplamak isteğe bağlıdır, çünkü hesaplanmış öznitelikler zaten projeye birlikte gelir.

```

1 pip install -r requirements.txt
2 make experiment # ~2 min, uses the committed feature tables
3 make dashboard # interactive replay at http://localhost:8501
4 make test
5 make data # optional: download + extract raw data (~1 GB / 6 GB)
6 make features # optional: rebuild the feature tables from raw data

```

Komutlar ne yapıyor? Satır 1 gerekli Python kütüphanelerini kurar. Satır 2 deneyin tamamını (dört dedektör, üç test, tüm tablo ve grafikler) depodaki hazır öznitelik tablolarıyla yaklaşık iki dakikada çalıştırır. Satır 3 paneli tarayıcıda açar. Satır 4 otomatik testleri çalıştırır. Satır 5 (isteğe bağlı) yaklaşık 1 GB’lık ham veriyi indirip 6 GB’a açar. Satır 6 (isteğe bağlı) öznitelik tablolarını ham veriden yeniden hesaplar.

8.4 Testler

Basitçe

Otomatik testler, kodun bir yerinde yapılan değişikliğin başka bir yeri sessizce bozmadığını kontrol eden küçük programlardır; bir arabanın her servis sonrası fren testinden geçmesi gibi. En önemli test, bilgisayarda yapay olarak “dış halkası hasarlı bir rulman sesi” üretir ve sistemin bunu dış halka hasarı olarak tanıyıp tanımadığını kontrol eder. Böylece cevabını bildiğimiz bir soruyla sistemi sınamış oluruz.

On iki otomatik test, sessizce bozulması en olası kısımları denetler. En önemlisi, gürültü içine gömülmüş ve tam olarak BPFO hızında tekrarlanan rezonans patlamalarından oluşan yapay bir dış bilezik hasarı üretir ve zarf özniteliğinin bunu tespit edip dış bileziğe atfettiğini kontrol eder (Kod 10). Diğer testler arıza frekanslarını yayımlanmış değerlerle, Gauss gürültüsünün istatistiklerini (RMS = 1, basıklık = 3), debounce ve kalıcılık mantığını, eşiğin puan kaymasına karşı değişmezliğini (bütün puanlar aynı miktarda kaydırılırsa alarm kararlarının değişmemesini) ve panelin her test için iki dedektörle hatasız açıldığını denetler.

Kod 10: Yapay arıza testi (`tests/test_core.py`’den alıntı).

```

1 def test_envelope_picks_up_outer_race_impacts():
2     """Synthetic outer-race defect: resonance bursts repeating at BPFO."""
3     rng = np.random.default_rng(1)
4     t = np.arange(FS) / FS
5     bpfo = fault_frequencies()["bpfo"]
6     impacts = np.zeros(FS)
7     impacts[(np.arange(0, 1, 1 / bpfo) * FS).astype(int)] = 1.0
8     ring = np.exp(-t[:200] * 2000) * np.sin(2 * np.pi * 4000 * t[:200])
9     faulty = np.convolve(impacts, ring)[:FS] * 3 + 0.3 * rng.standard_normal(FS)
10    healthy = 0.3 * rng.standard_normal(FS)
11    f_faulty, f_healthy = snapshot_features(faulty), snapshot_features(healthy)
12    assert f_faulty["env_bpfo"] > 5 * f_healthy["env_bpfo"]
13    assert f_faulty["env_bpfi"] > f_faulty["env_bpfi"]
14    assert f_faulty["kurtosis"] > f_healthy["kurtosis"]

```

Kod ne yapıyor? Satır 3–4 sabit tohumlu bir rastgele sayı üretici ve bir saniyelik bir zaman eksenini hazırlar. Satır 5–7 BPFO’yu hesaplar ve bir saniye içinde, tam BPFO ritminde (saniyede yaklaşık 236 kez) birer “tık” yerleştirir. Satır 8 tek bir tıknın sesini taklit eder: 4 kHz’de çınlayıp hızla sönen bir titreşim (çekiçle vurulan bir metalin sesi gibi). Satır 9 tıkları bu çınlamayla birleştirir ve üstüne rastgele gürültü ekler: bu “hasarlı rulman”dır. Satır 10 yalnızca gürültüden oluşan “sağlıklı rulman”ı üretir. Satır 11 ikisinin özniteliklerini hesaplar. Satır 12–14 üç şartı kontrol eder: hasarlı sinyalin dış bilezik zarf enerjisi sağlıklıdakinin en az 5 katı olmalı; hasarlı sinyalde dış bilezik enerjisi iç bilezikten büyük olmalı (yani teşhis doğru parçayı göstermeli); hasarlı sinyalin basıklığı daha yüksek olmalı (darbeler yakalanmalı). Şartlardan biri bile tutmazsa test başarısız olur ve bir şeylerin bozulduğu hemen anlaşılır.

8.5 Sürüm kontrolü

Basitçe

Sürüm kontrolü, projedeki her değişikliğin kimin tarafından, ne zaman ve neden yapıldığını kaydeden bir sistemdir; bir belgenin tüm eski sürümlerini saklayan bir “değişiklik geçmişi” gibi. Yeni özellikler ana sürümü bozmamak için ayrı bir kopyada (dal) geliştirilip, hazır olunca ana sürüme eklenmiştir.

Proje GitHub’da barındırılmaktadır (https://github.com/ecemaydemir/bearing_early_fault_warning). İlk veri işleme hattı doğrudan main dalına eklenmiş; sonraki eklemeler (panel, bu rapor) ayrı dallarda geliştirilip pull request’lerle (değişikliğin gözden geçirilip ana sürüme eklenmesi talebiyle) birleştirilmiştir.

9 Tartışma ve Sınırlamalar

Basitçe

Her çalışmanın sınırları vardır ve bunları açıkça söylemek sonuçlara ne kadar güvenebileceğimizi anlamak için önemlidir. Bizimkiler: (1) bozulmanın ne zaman başladığını kesin bilmiyoruz, sadece test sonunda bozuk olduğunu biliyoruz; (2) yalnızca üç deney var, bu yüzden “şu yöntem kesin en iyisi” diyemeyiz; (3) makine durdurulup çalıştırıldığında normal değişiyor ve sistem buna uyum sağlamıyor; (4) hasar yeri tahmini iç halka ve makara hasarlarında henüz zayıf; (5) alarmin sona kadar susmamasını şart koşmamız bazı gerçek erken uyarıları saymamamıza yol açıyor.

- **Gerçek etiket (ground truth).** Yalnızca “test sonunda arızalıydı” bilgisi vardır. Bu nedenle uyarı süresi, bilinen bir başlangıcın tespitini değil bir alarmin ne kadar erken kalıcı hâle

geldiğini ölçer; yanlış alarm için kullanılan sağlıklı pencereler belgelenmiş birer varsayımdır.

- **Küçük örneklem.** Üç deney ve dört arızalı rulman, istatistiksel olarak sağlam sıralamalar için yeterli değildir; sayılar başka ayarlarla değişecektir. Ayrılmış testlerin sonuçlarını dürüst tutmak için ayarlar yalnızca test 2 üzerinde seçilmiştir.
- **Durağan olmayan çalışma.** Test 3'teki yeniden başlatmalar her rulmanın temel seviyesini kaydırır. Gerçek bir tesiste temel seviye, bakımdan sonra veya bir operatör değişikliğinin zararsız olduğunu onayladığında yeniden kestirilir.
- **Dar bantlı teşhis.** İç bilezik ve yuvarlanma elemanı hasarları mil ve kafes dönüşüyle modüle edilir (darbelerin şiddeti dönüşle birlikte düzenli olarak artıp azalır); yan bantları hesaba katan veya spektral basıklığa (darbelerin en belirgin olduğu frekans bandını otomatik bulan bir yöntemle) dayalı bant seçimi teşhisi daha genel hâle getirir.
- **Kalıcılık tanımı temkinlidir.** Alarmin sona kadar sürmesini şart koşturmak, aralıklı ama gerçek erken uyarıları (test 1, rulman 3) dışarıda bırakır; uygulamada aralıklı bir alarm da operatör için değerli bir bilgidir.

10 Sonuç ve Gelecek Çalışmalar

Basitçe

Bu projede, bir makinenin sağlıklı sesini öğrenip bir rulman bozulmaya başladığında haber veren bir “elektronik doktor” kurduk. Sisteme hiç arıza örneği göstermedik; sadece ilk günlerdeki normal titreşimi öğrettik. En temiz deneyde sistem, rulman kırılmadan yaklaşık üç gün önce alarm verdi ve hasarın rulmanın dış halkasında olduğunu doğru söyledi. Daha zor deneylerde, çok erken uyarı yapmak istediğimizde daha sık boşuna alarm aldığımızı, alarmı seyreklettiğimizde ise geç kaldığımızı gördük. En büyük zayıflık, makine durdurulup yeniden çalıştırıldığında sistemin yeni normali bir sorun sanmasıdır. Sonraki adım, sistemin bu tür olaylardan sonra “normal”i kendiliğinden güncellemesini ve hasarın yerini her tür hasar için doğru söyleyebilmesini sağlamaktır.

Proje, arıza etiketi kullanmayan, çalışan ve yeniden üretilebilir bir rulman erken uyarı hattı ortaya koymaktadır. En temiz deneyde üç gün (deneyin %45'i) önceden uyarır ve doğru hasarı belirtir; daha zor deneylerde ise uyarı süresi ile yanlış alarm arasındaki ödünleşimi açık hâle getirir ve nerede, neden başarısız olduğunu belgeler. En değerli sonraki adımlar şunlardır:

1. yeniden başlatma ve bakımdan sonra uyarlanabilir temel seviye kestirimi (test 3);
2. alarmdan sonra kalan faydalı ömür (RUL; rulmanın tamamen bozulmasına ne kadar süre kaldığı) kestirimi; örneğin sağlık göstergesine bir bozulma eğilimi uydurarak;
3. iç bilezik ve yuvarlanma elemanı teşhisi için yan bantları hesaba katan zarf öznelilikleri veya spektral basıklık;
4. erken uyarıyı az yanlış alarmla birleştirmek için bir topluluk alarmı (ör. dört dedektörden ikisi aynı anda alarm verdiğinde çalan bir alarm);
5. başka arızaya kadar çalıştırma veri setlerinde (ör. FEMTO/PRONOSTIA, XJTU-SY) doğrulama.

A Sözlük: Temel Kavramlar

Bu sözlük, raporda geçen teknik terimleri hiçbir ön bilgi gerektirmeden açıklar. Terimler konularına göre gruplanmıştır.

Makine ve rulman

Rulman

Dönen bir mili taşıyan ve dönmesini kolaylaştıran parça. İki halka ve aralarında yuvarlanan küçük bilyeler ya da silindirlerden oluşur. Bisikletin tekerlek göbeğinde ve arabanın tekerleklerinde de bulunur.

İç bilezik

Rulmanın içteki halkası; mile takılıdır ve onunla birlikte döner.

Dış bilezik

Rulmanın dıştaki halkası; makinenin gövdesine sabitlenmiştir ve dönmez.

Yuvarlanma elemanı

İki halka arasında yuvarlanan bilye ya da makara (küçük silindir). Sürtünmeyi azaltan, kayma yerine yuvarlanmayı sağlayan parçalardır.

Kafes

Yuvarlanma elemanlarını eşit aralıklarla tutan, birbirlerine çarpmalarını önleyen çerçeve; bir dönme dolaptaki kabinleri birbirine bağlayan iskelet gibi.

Run-to-failure (arızaya kadar çalıştırma)

Bir makinenin, bir parçası bozulana kadar bilerek durdurulmadan çalıştırıldığı deney. Böylece bozulmanın baştan sona nasıl geliştiği kaydedilir.

Rejim değişimi

Makinenin çalışma koşulunun kalıcı olarak biraz değişmesi (ör. durdurulup yeniden çalıştırıldıktan sonra parçaların biraz farklı oturması). Arıza değildir ama “normal” sesin değişmesine yol açar; yeni bir eve taşınınca evin seslerinin farklı gelmesi gibi.

Titreşim ve ölçüm

Titreşim

Bir cismin hızla ileri geri salınması. Çalışan her makine titreşir; titreşimin biçimi makinenin sağlığı hakkında bilgi taşır, tıpkı bir kalp sesinin kalp hakkında bilgi taşıması gibi.

İvmeölçer

Titreşimi ölçen küçük sensör. Telefonunuzun ekranı döndürdüğünüzde dönmesini sağlayan sensörün daha hassas bir benzeridir.

g birimi

İvmenin birimi; 1 g, yerçekiminin bir cisme kazandırdığı ivmedir. 0,1 g, sensörün yerçekiminin onda biri kadar bir ivmeyle sarsıldığı anlamına gelir.

Hz (hertz)

“Saniyede kaç kez” anlamına gelen birim. 50 Hz saniyede 50 kez, 1 kHz (kilohertz) saniyede 1 000 kez demektir.

Örnekleme frekansı

Sensörün saniyede kaç ölçüm aldığı. Bu projede yaklaşık 20 000 (20 kHz); bir filmin saniyedeki kare sayısına benzer: ne kadar yüksekse o kadar hızlı olaylar yakalanır.

Snapshot / kayıt

Her 10 dakikada bir alınan bir saniyelik titreşim ölçümü; makinenin o anki “ses kaydı”. Her kayıttaki sensör başına 20 480 sayı vardır.

Frekans

Bir şeyin ne sıklıkla tekrarlandığı. Seste frekans tonun inceliğini belirler: düşük frekans bas, yüksek frekans tiz sestir.

Spektrum

Bir sinyalin hangi frekanslarda ne kadar enerji taşıdığını gösteren döküm; bir müzik çalarının ekolayzır çubukları gibi. “Bu ses hangi tonlardan oluşuyor?” sorusunun cevabıdır.

FFT (Hızlı Fourier Dönüşümü)

Zamanla değişen bir sinyali spektrumuna çeviren hızlı hesap yöntemi. Bir akoru dinleyip hangi notalardan oluştuğunu söylemek gibidir.

Rezonans

Bir cismin vurulduğunda kendi doğal tonunda çınlaması; bir zilin veya bardağın tınlaması gibi. Rulman gövdesi de her darbeye birkaç kHz’lik tonlarda çınlar.

Bant geçiren filtre

Bir sinyalden yalnızca belirli bir frekans aralığını geçirip gerisini süzen işlem. Radyoda tek bir istasyona ayar yapmak gibi.

Zarf (envelope)

Hızlı salınan bir sinyalin tepelerini birleştiren dış hat; sesin “kabarıp sönmesi”. Hızlı çınlamaların içinde gizlenmiş yavaş ritmi ortaya çıkarır.

Hilbert dönüşümü

Bir sinyalin zarfını hesaplamak için kullanılan matematiksel araç. Bu raporda tek işlevi “dış hattı çizmek”tir.

BPFO / BPFI / BSF / FTF

Rulmanın dört “hasar ritmi”: dış bilezikteki bir çukurun (BPFO), iç bilezikteki bir çukurun (BPFI), bir makaradaki çukurun (BSF) ve kafesteki bir sorunun (FTF) titreşimde yarattığı tekrarlanma hızları. Bu projedeki rulman için sırasıyla yaklaşık 236, 297, 140 ve 15 Hz’dir. Hangi ritmin belirttiği, hasarın hangi parçada olduğunu söyler.

Harmonik

Bir frekansın tam katları. 236 Hz’in harmonikleri 473, 709, ... Hz’dir. Gerçek darbeler tek bir ton değil, bu katlardan oluşan bir “aile” üretir.

Özet sayılar (öznitelikler)**Öznitelik (durum göstergesi)**

Ham sinyalden hesaplanan, makinenin durumunu özetleyen tek bir sayı. Kan tahlilindeki şeker veya kolesterol değerleri gibi.

RMS

Sinyalin ortalama “gücü”. Titreşim büyüdükçe artar; bir sesin genel yüksekliği gibi.

Tepe değeri

Kayıttaki en büyük tek sapma; en sert “vuruş”.

Basıklık (kurtosis)

Sinyalde seyrek ama sert sıçramalar olup olmadığını ölçer. Düzgün bir hışırtı için 3’tür; arada bir gelen keskin “tık”lar onu hızla büyütür.

Çarpıklık

Sinyalin bir yöne (yukarı ya da aşağı) diğerinden daha fazla sapıp saptığı; dengesizlik ölçüsü.

Crest (tepe) faktörü

En sert vuruşun ortalama güce oranı: “en büyük sıçrama, ortalamanın kaç katı?”

Logaritma

Sayıları “kaçıncı 10’un katı” biçiminde ölçen işlem. 10, 100 ve 1000’i eşit adımlara dönüştürür; çok farklı büyüklükleri aynı grafikte göstermeye ve uç değerlerin etkisini azaltmaya yarar. Logaritmik eksende eşit aralıklar eşit *katlara* karşılık gelir.

Standartlaştırma / z-skoru

Bir değeri “normalin ortalamasından kaç standart sapma (tipik dalgalanma) uzakta?” biçiminde ifade etme. $z = 0$ tam normal, $z = 3$ “normalin çok üstünde” demektir. Bir öğrencinin notunu sınıf ortalamasına göre ifade etmek gibidir.

Kovaryans

Birden fazla ölçümün birlikte nasıl değiştiğini (biri artınca diğeri de artıyor mu?) ve her birinin ne kadar dalgalandığını özetleyen tablo.

Öğrenme ve tespit**Gözetimsiz öğrenme**

Bilgisayara doğru cevapların gösterilmediği öğrenme. Bu projede sisteme “bu arıza” diye hiçbir örnek gösterilmez; sistem yalnızca normalin neye benzediğini öğrenir.

Eğitim / kalibrasyon / test dönemi

Her deneyin zamana göre üçe bölünmesi. Eğitim (ilk %25): sistem normali öğrenir. Kalibrasyon (sonraki %10): alarm sınırı belirlenir. Test (geri kalanı): sistem gerçekten sınanır. Bir öğrencinin ders çalışması, deneme sınavı ve asıl sınavı gibi.

Anomali puanı

Bir kaydın “normale ne kadar benzemediğini” anlatan sayı; ne kadar büyükse o kadar tuhaf.

Eşik

Puanın üstüne çıkınca “bu normal değil” dediğimiz sınır; ateş için 38 derece gibi.

Kuantil

Bir sayı kümesinde belirli bir oranın altında kaldığı değer. 0,995 kuantili, değerlerin %99,5’inin altında kaldığı sayıdır; “neredeyse en yüksek değer”. Medyan 0,5 kuantilidir (tam orta değer).

Mahalanobis uzaklığı

Yeni bir ölçümün sağlıklı ölçümler bulutunun merkezinden ne kadar uzak olduğunu, ölçümlerin normal dalgalanmasını ve birbiriyle ilişkisini hesaba katarak veren uzaklık. Boyu uzun birinin ağır olması olağandır, uzun ama çok hafif olması dikkat çeker.

Ledoit-Wolf

Az veriyle kovaryans tablosu hesaplarken onu daha kararlı hâle getiren bir “yumuşatma” yöntemi; az örnekten aşırı uç sonuçlar çıkmasını önler.

Isolation Forest

Rastgele sorularla gözlemleri birbirinden ayırmaya çalışan yüzlerce “karar ağacı”ndan oluşan yöntem. Kalabalıktan az soruyla ayrılabilen gözlem tuhaftır; turuncu kostümlü birini kalabalıkta bulmak gibi.

Sinir ağı

Beyindeki sinir hücrelerinden esinlenen, çok sayıda ayarlanabilir sayıdan oluşan bir hesaplama yapısı. Örneklere bakarak bu sayıları kendisi ayarlar ve böylece “öğrenir”.

LSTM

Sıralı veriyi (ör. zaman içinde gelen ölçümleri) okurken önceki adımları hatırlayabilen bir sinir

ağı türü; bir cümleyi okurken baştaki kelimeleri unutmayan bir okuyucu gibi. Adı “Uzun Kısa Süreli Bellek”tir.

Otokodlayıcı

Girdisini kısa bir özete sıkıştırıp sonra bu özetten girdiyi yeniden oluşturmayı öğrenen sinir ağı. Yalnızca normal veriyle eğitilirse, tanımadığı (anormal) veriyi kötü yeniden oluşturur; bu “beceriksizlik” anomali puanı olarak kullanılır.

Gürültü giderici otokodlayıcı

Eğitim sırasında girdisine bilerek biraz parazit eklenen ve temiz hâlini çıkarması istenen otokodlayıcı. Ezberlemek yerine verinin genel yapısını öğrenmeye zorlanır.

Epoch (epok)

Eğitim verisinin baştan sona bir kez gözden geçirilmesi. 60 epok, verinin 60 kez çalışılması demektir; bir kitabı 60 kez okumak gibi.

Geliştirme kümesi / ayrılmış küme

Geliştirme kümesi (test 2), yöntemi kurarken ve ayarlarken kullanılan veridir. Ayrılmış küme (test 1 ve test 3), ayarlar bitene kadar hiç bakılmayan ve yalnızca son değerlendirmede kullanılan veridir; öğrencinin önceden görmediği sınav soruları gibi.

Veri sızıntısı

Değerlendirme sırasında sistemin, gerçekte bilemeyeceği bir bilgiyi (ör. gelecekteki veriyi) yanlışlıkla görmüş olması. Sınav sorularının önceden sızması gibi; sonuçlar olduğundan iyi görünür.

Alarm ve değerlendirme

k-of-n kuralı

“Son n ölçümün en az k tanesi eşiği aşarsa alarm ver” kuralı. Burada son 6 ölçümün 4’ü. Tek bir tuhaf ölçüme kapılmamayı sağlar; tek bir öksürük için değil, öksürük sürerse doktora gitmek gibi.

Debounce

Kısa süreli, gelip giden sinyallerin alarmı açıp kapatmasını önleyen sakinleştirme işlemi; k-of-n kuralı bunun bir yoludur.

Alarm epizodu

Alarmın kesintisiz olarak açık kaldığı bir zaman dilimi; alarmın bir kez çalmaya başlayıp susmasına kadar geçen süre.

Kalıcı alarm

Bir kez başladıktan sonra testin sonuna kadar hiç susmayan alarm epizodu. Uyarı süresi bunun başlangıcından ölçülür.

Uyarı süresi (lead time)

Kalıcı alarm ile rulmanın bozulup testin durdurulması arasında geçen süre. Ne kadar uzunsa bakım için o kadar hazırlık zamanı vardır.

Yanlış alarm

Makine hâlâ sağlıklıyken verilen alarm; ekmek kızartırken öten duman dedektörü gibi. Çok sık olursa insanlar alarma güvenmemeye başlar.

Rulman-saat

İzlenen rulman sayısı ile izleme süresinin çarpımı. Dört rulmanı 10 saat izlemek 40 rulman-saattir. Yanlış alarm sayısını bu birime bölmek, farklı uzunluktaki testleri adil karşılaştırmayı sağlar.

Yazılım

Streamlit

Python ile hızlıca etkileşimli web sayfaları yapmayı sağlayan bir kütüphane. Bu projenin panelini oluşturmak için kullanılmıştır.

Panel (dashboard)

Önemli bilgileri tek bakışta gösteren ekran; bir arabanın gösterge paneli gibi. Burada her rulmanın durumunu, puanları ve alarmları gösterir.

Kaynaklar

- [1] J. Lee, H. Qiu, G. Yu, J. Lin, and Rexnord Technical Services. *Bearing Data Set*. IMS, University of Cincinnati. NASA Prognostics Data Repository, NASA Ames Research Center, 2007.
- [2] H. Qiu, J. Lee, J. Lin, and G. Yu. Wavelet filter-based weak signature detection method and its application on rolling element bearing prognostics. *Journal of Sound and Vibration*, 289(4-5):1066-1090, 2006.
- [3] R. B. Randall and J. Antoni. Rolling element bearing diagnostics—A tutorial. *Mechanical Systems and Signal Processing*, 25(2):485-520, 2011.
- [4] F. T. Liu, K. M. Ting, and Z.-H. Zhou. Isolation forest. In *Proc. IEEE International Conference on Data Mining*, pages 413-422, 2008.
- [5] O. Ledoit and M. Wolf. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2):365-411, 2004.
- [6] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735-1780, 1997.
- [7] P. Malhotra et al. LSTM-based encoder-decoder for multi-sensor anomaly detection. *ICML Anomaly Detection Workshop*, 2016.